

RAVEL: Reversible SLO-Aware Virtual Placement for Geo-Distributed LLM Serving

Abstract

Regional capacity constraints and rising LLM inference demand are driving providers to pool GPU resources across geographically distributed clusters. Cross-region scheduling must navigate differences in WAN latency and regional serving conditions. Together, these regional differences and heterogeneous request-level SLOs make placement feasibility request-specific, so a region that is optional for one request may be the only SLO-feasible choice for another. When available capacity is limited, immediately finalizing placement may leave no SLO-feasible capacity for later arrivals. Waiting reveals more demand but leaves less time to meet the SLO. Conventional cross-region schedulers make a one-shot placement for each request using only the demand visible at arrival. Later arrivals cannot revise earlier placements, even when the affected requests have not yet entered the serving engine. The key observation is that every arrival can enter capacity planning immediately while placement remains reversible until engine handoff. We present RAVEL, an SLO-aware serving system that realizes this separation through tentative placement, immediate demand accounting, and bounded pre-handoff replanning. On a vLLM deployment with five serving replicas across three clusters connected by physical WAN paths, RAVEL improves request-level SLO attainment over the best baseline by 8.8, 11.2, and 13.4 percentage points on LMSYS, Burst, and DeepResearch, respectively, while reducing P95 TTFT by 49.1–80.5% and P95 end-to-end latency by 15.5–56.8%, at $16\times$ load.

1 Introduction

Geo-distributed LLM serving expands available GPU capacity, but not every region is equally useful to every request. Large-scale private WANs expose heterogeneous inter-region paths [12, 14], while distributed inference systems must additionally contend with heterogeneous accelerator, serving, and network conditions [5, 27, 35, 44, 49]. Together with heterogeneous request-level SLOs, these differences make regional ca-

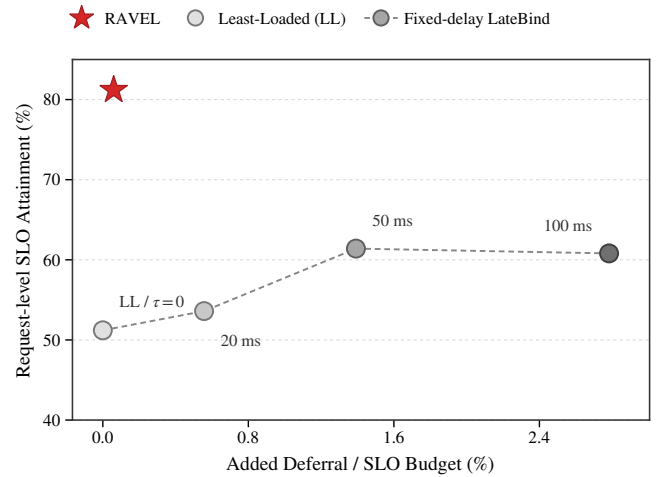


Figure 1: **Information–slack tradeoff.** Fixed-delay late binding consumes SLO slack to observe more demand; RAVEL accounts demand immediately and selectively uses a request-specific, slack-bounded mobility hold for pre-handoff replanning.

capacity request-dependent: a region that is merely attractive for one request may be the only SLO-feasible choice for another. A placement therefore determines not only GPU load and cache locality, but also the WAN path and remote serving conditions experienced by the request. Interactive requests may impose tight time-to-first-token (TTFT) and time-between-token (TBT) bounds, whereas longer-running or multi-stage requests may instead be governed by end-to-end completion or quality-of-experience objectives [23, 25, 45, 57, 59]. Consequently, allocating regional capacity well requires reasoning about both current serving conditions and the SLO-feasible alternatives available to each request.

Existing distributed routers improve *where* an arriving request should execute using signals such as serving load, cache locality, WAN conditions, request-level SLOs, and adaptive routing feedback [20, 25, 40, 44, 49, 54]. However, these routing abstractions largely couple destination selection with com-

mitment: once a request is assigned using the currently visible state, later arrivals generally do not influence that placement before execution begins. This coupling creates an additional online allocation problem. A locally attractive placement for the current request can consume SLO-feasible capacity that a later, more region-constrained request cannot replace. The missing question is therefore not only *where should this request execute?*, but also *when should an earlier placement become irreversible?*

A two-request example shows why early commitment can waste SLO-feasible capacity. Consider two serving regions, *A* and *B*. The first request, r_1 , can satisfy its SLO in either region, whereas a later request, r_2 , can succeed only in *A*. When r_1 arrives first, *A* may appear faster and therefore be selected by a locally optimal router. If r_2 arrives shortly afterward, however, this decision has consumed its only feasible capacity. Placing r_1 in *B* instead would allow both requests to succeed: *region A is merely faster for r_1 , but it is essential for r_2* . Thus, optimizing requests independently can reduce aggregate request-level SLO attainment even when each individual placement appears reasonable when it is made.

Delaying commitment can reveal the second request, but observation consumes the same SLO slack needed for WAN transfer, queueing, and inference. A fixed-delay late-binding policy can therefore observe more demand before placing a request, yet every millisecond of added deferral also reduces the time remaining to satisfy that request’s SLO. We call this tension the *information–slack tradeoff*. Figure 1 illustrates it empirically: a short fixed deferral can expose useful arrivals and improve SLO attainment, but the benefit saturates as the deferral consumes an increasing fraction of request SLO budgets. This leaves a design opportunity between immediate one-shot placement and fixed-delay late binding: make arrived demand visible immediately while preserving only bounded, request-specific placement revisability.

RAVEL’s key insight is that deferred commitment does not require deferred accounting. Rather than treating placement as one atomic decision, RAVEL separates when a request begins influencing capacity planning from when its destination becomes final. On arrival, RAVEL obtains replica-specific SLO quotes, selects a tentative serving replica, and immediately attributes the request’s prospective demand to that replica. Later arrivals therefore observe earlier demand even while the corresponding placements remain revisable. While an eligible request remains Router-owned, newly revealed demand may trigger bounded replanning and transfer both its tentative ownership and prospective charge to another replica. The placement becomes final only after successful engine handoff. By stopping reassignment at this explicit boundary, RAVEL preserves pre-handoff flexibility without migrating engine-managed KV state.

Realizing this separation safely and efficiently requires three capabilities. First, RAVEL must determine which replicas remain SLO-feasible under heterogeneous WAN delays,

engine backlog, Prefill and Decode interference, and request-specific success criteria. It addresses this challenge with calibrated, replica-specific SLO quotes derived from live serving state. Second, tentative work must affect subsequent decisions without becoming double-counted or stale after reassignment. RAVEL therefore maintains transferable prospective accounting whose ownership and replica-dependent charges move atomically with each tentative placement. Third, revisability must not require repeatedly optimizing over every outstanding request or delaying work indefinitely. RAVEL restricts expensive replanning to a bounded Router-owned cohort and ends all placement revision at the engine-handoff boundary. Together, these mechanisms retain useful online flexibility while bounding the latency and state-management costs introduced by the control plane.

We implement RAVEL as an asynchronous routing layer for vLLM [15] and evaluate it using five serving replicas across three clusters connected by physical WAN paths. Our experiments cover conversational traffic, bursty arrivals, and completion-oriented demand over six offered-load levels. At $16\times$ load, RAVEL improves request-level SLO attainment over DualMap by **8.8, 11.2, and 13.4 percentage points** on LMSYS, Burst, and DeepResearch, respectively. At the same load, it reduces P95 TTFT by **49.1–80.5%** relative to DualMap. The online replanning path remains millisecond-scale, with a measured P95 scheduling-pass latency of 2.10 ms at Burst $12\times$ load.

In summary, this paper makes three contributions:

- We identify the *request-dependent value of regional capacity* and the resulting *information–slack tradeoff*: committing too early ignores demand revealed by later arrivals, whereas waiting too long consumes the SLO slack needed for successful execution.
- We introduce an *immediate-accounting, deferred-commitment* abstraction and realize it through replica-specific SLO quotes, transferable prospective accounting, bounded pre-handoff replanning, and an explicit engine-handoff commitment boundary.
- We implement RAVEL on vLLM and evaluate it on a physical-WAN deployment, demonstrating consistent SLO-attainment improvements across conversational, bursty, and completion-oriented workloads while retaining millisecond-scale online planning overhead.

2 Problem and Design Opportunity

The design opportunity arises from separating three decisions that request routing often couples: *placement*, *regional demand accounting*, and *commitment*. A placement associates a request with a candidate serving replica and, therefore, with the region containing that replica. Regional demand accounting attributes an arrived request’s prospective work to its

tentative replica and region so that the request influences subsequent placement decisions. Commitment is the boundary after which the routing layer no longer revises that placement. Using only causally available information, the router seeks to maximize the fraction of requests that satisfy all applicable request-level SLOs. RAVEL chooses successful handoff to the serving engine as its commitment boundary: a request remains reassignable while it is Router-owned and becomes final only after handoff. This is a deliberate control-plane boundary rather than an inherent irreversibility property of LLM execution; it preserves placement flexibility without requiring migration of engine-managed KV state.

2.1 Request-Dependent Regional Feasibility

Prior SLO-aware serving systems show that whether a request can meet its latency objective depends on both request-specific requirements and evolving serving conditions [23, 25, 57, 59]. RAVEL makes this dependence explicit at the geographic placement layer: SLO feasibility is specific to a request, candidate replica, and decision time rather than a static property of a region.

Let c denote a serving region, $\mathcal{R}_c$ the replicas in that region, and $r \in \mathcal{R}_c$ a candidate replica. For request i , let $\Phi_{i,r}(t)$ indicate whether replica r is predicted, using only information available at time t , to satisfy every SLO constraint applicable to i . This prediction depends on the request’s SLO semantics and remaining slack, its estimated service demand, the WAN path to region c , and the currently observed serving state at replica r . Because both serving state and remaining slack evolve over time, $\Phi_{i,r}(t)$ is an online feasibility estimate rather than a guarantee: future arrivals and prediction error may still change the realized outcome. Section 3.2 describes how RAVEL constructs these replica-level predictions.

We define the feasible-region set of request i as

$$\mathcal{F}_i(t) = \{c \mid \exists r \in \mathcal{R}_c : \Phi_{i,r}(t) = 1\}. \quad (1)$$

The time index captures two distinct sources of change. First, new arrivals and accounting updates alter the serving state visible to the router, so $\mathcal{F}_i(t)$ may expand or shrink as contention changes. Second, even if network and serving conditions remain unchanged, elapsed time consumes TTFT or completion slack and can therefore remove regions from the feasible set.

The size and composition of $\mathcal{F}_i(t)$ characterize how substitutable regional capacity is for request i . A request with several feasible regions has multiple serving alternatives, whereas a request with a small feasible set depends on comparatively scarce regional capacity. For example, if $\mathcal{F}_1 = \{A, B\}$ and $\mathcal{F}_2 = \{A\}$, assigning request 1 to region A may consume the only feasible option available to request 2, even though request 1 could instead execute in B . Thus, the opportunity cost of assigning capacity in a region depends on which requests can substitute away from that region. Regional capacity is therefore request-dependent, and optimizing request-

Table 1: Demand-visibility and placement-finality operating points. One-shot couples both decisions at arrival, LateBind- τ delays both, and RAVEL exposes demand immediately while deferring only commitment.

Policy	Demand visible	Placement final
One-shot	At arrival	At routing
LateBind- τ	After τ	After τ
RAVEL	At arrival	At engine handoff

region assignments independently need not maximize aggregate request-level SLO attainment.

2.2 The Information–Slack Tradeoff

The allocation problem is online: the router must make decisions without observing future arrivals. A placement that is reasonable under the currently visible state may become undesirable after a more region-constrained request arrives. This setting shares concerns with low-latency online and shared-state cluster scheduling, where decisions must be made from incomplete and continually changing information [11, 17, 31, 37]. RAVEL does not assume an arrival oracle or attempt to reproduce an offline optimal allocation; instead, it preserves bounded revisability using only causally observed requests and serving state.

The relevant operating points differ along two independent axes: when arrived demand becomes visible to capacity planning and when placement becomes final. Table 1 separates these decisions. A one-shot operating point assigns and accounts for a request immediately and does not subsequently revise that placement. At the other extreme, our diagnostic LateBind- τ policy keeps a request centrally unassigned and unaccounted for a fixed interval τ , then observes the current state and makes a single final placement. RAVEL occupies the remaining operating point: it makes the request’s demand visible immediately while retaining placement revisability until engine handoff.

Deferring commitment can reveal information about later demand, but that information is not free under request-level SLOs. Delay scheduling similarly shows that waiting can expose better placement opportunities [55], while deadline-driven scheduling emphasizes the cost of consuming finite execution slack [22]. Tail-sensitive distributed services further motivate treating latency slack as a scarce resource rather than an unlimited observation window [7]. In geo-distributed LLM serving, these effects interact directly: holding request i for τ consumes τ of its available TTFT or completion slack, leaving less time for WAN transfer, queueing, and inference. A longer delay may therefore reveal more subsequent demand while simultaneously reducing the set of regions in which the held request can still succeed. Equivalently, $\mathcal{F}_i(t)$ can shrink

purely because the router waits, even without an increase in serving load.

We refer to this tension as the *information–slack tradeoff*. Figure 1 illustrates it empirically: short fixed deferral can improve SLO attainment by exposing useful arrivals, but the benefit saturates and eventually reverses as waiting consumes an increasing fraction of request SLO budgets. The design opportunity is therefore not simply to delay placement longer. It is to make arrived demand visible immediately while deferring only placement finality.

Exploiting this opportunity requires preserving useful revisability without introducing unbounded waiting, inconsistent accounting, or unsafe ownership transitions. A practical design must (i) estimate request-specific SLO feasibility, (ii) account each tentative request exactly once and transfer its prospective charge when placement changes, (iii) bound both deliberate observation and multi-request replanning, and (iv) enforce a failure-safe commitment boundary at engine handoff. Section 3 describes how RAVEL satisfies these requirements. ““

3 RAVEL Design

3.1 Overview

RAVEL realizes immediate accounting and deferred commitment through two ownership states separated by successful engine handoff. While Router-owned, each request has exactly one tentative replica owner and one set of prospective charges; after handoff, its placement is final and is no longer eligible for Router-level reassignment. This ownership boundary lets RAVEL exploit newly observed demand without migrating engine-managed KV state.

Figure 2 summarizes the request lifecycle. On arrival, RAVEL constructs request-specific SLO quotes for all candidate replicas, selects an initial virtual placement, and immediately accounts the request’s prospective demand at that replica. As later arrivals and engine feedback change the visible state, RAVEL may replan a bounded cohort of still-Router-owned requests. A reassignment transfers both tentative queue ownership and replica-dependent prospective charges. Requests cross a shallow, capacity-gated dispatch frontier before handoff. A rejected handoff restores Router ownership, whereas successful handoff transfers ownership to the serving engine and makes placement final.

The design maintains five invariants. First, each Router-owned request has exactly one tentative owner. Second, its prospective demand is represented exactly once and follows any reassignment. Third, only Router-owned requests may change placement. Fourth, replanning uses only causally observed requests and serving state and is bounded in both observation time and cohort size. Finally, successful engine handoff is the only commitment event. The following subsections describe replica quotes, prospective accounting, bounded re-

planning, unified placement and reversible dispatch, and the handoff boundary that enforces these invariants.

““latex

3.2 Replica SLO Quotes

Every placement decision consumes a *replica SLO quote*: a causal, request-specific summary of whether a candidate replica is predicted to satisfy the request’s SLO and of the cost of serving it there. Quotes provide a common placement interface across heterogeneous request semantics without collapsing TTFT, TBT, and completion constraints into an arbitrary weighted score. Prior phase-aware serving systems show that Prefill and Decode have distinct resource demands and can interfere under shared execution [3, 13, 32, 59]. RAVEL incorporates these effects into replica-specific feasibility estimates rather than introducing a new phase scheduler.

Let $c \in \mathcal{C}$ denote a serving cluster, $\mathcal{R}_c$ its replicas, $c(r)$ the cluster containing replica r , and $\mathcal{R} = \bigcup_c \mathcal{R}_c$. For request i and candidate replica r , RAVEL constructs

$$Q_{i,r}(t) = \left(\Phi_{i,r}(t), \hat{T}_{i,r}^{\text{obj}}(t), \mathbf{z}_{i,r}(t) \right), \quad (2)$$

where $\Phi_{i,r}(t)$ indicates whether all SLO constraints applicable to request i are predicted to hold, $\hat{T}_{i,r}^{\text{obj}}(t)$ is the request-class-specific predicted objective used for ordering and slack calculations, and $\mathbf{z}_{i,r}(t)$ contains secondary capacity-pressure signals used by the placement policy. The quote depends only on information causally available at time t and is an online prediction rather than a formal SLO guarantee.

RAVEL combines calibrated cluster-level service profiles with replica-local live and prospective state. At active Decode concurrency n , it models Prefill service for a prompt of size p as $P_{c,n}(p) = b_{c,n} + a_{c,n}p$ and uses $d_c(n)$ for the calibrated post-first-token Decode time per token. For request i and candidate replica r , let $\hat{S}_{i,r}^{\text{pre}}$ denote its predicted Prefill-side service delay. This estimate includes live engine Prefill debt, Router-owned prompt work ahead of i , the request’s own prompt demand, and tentative assignments selected earlier in the current planning pass. RAVEL then predicts

$$\widehat{\text{TTFT}}_{i,r}(t) = (t - a_i) + \rho_{o_i,c(r)} + \hat{S}_{i,r}^{\text{pre}}, \quad (3)$$

where a_i is the request arrival time and $\rho_{o_i,c(r)}$ is the directed WAN delay from request origin o_i to cluster $c(r)$. The elapsed term $(t - a_i)$ directly charges time already spent under Router ownership against the remaining TTFT budget. Thus, even with unchanged WAN and serving conditions, a previously feasible replica may become infeasible as observation consumes request slack.

For a LATENCY request with TTFT and TBT bounds D_i^{TTFT} and D_i^{TBT} , RAVEL evaluates the two constraints independently:

$$\Phi_{i,r}^{\text{lat}}(t) = \mathbf{1} \left[\widehat{\text{TTFT}}_{i,r}(t) \leq D_i^{\text{TTFT}} \wedge d_{c(r)}(m_{i,r}) \leq D_i^{\text{TBT}} \right], \quad (4)$$

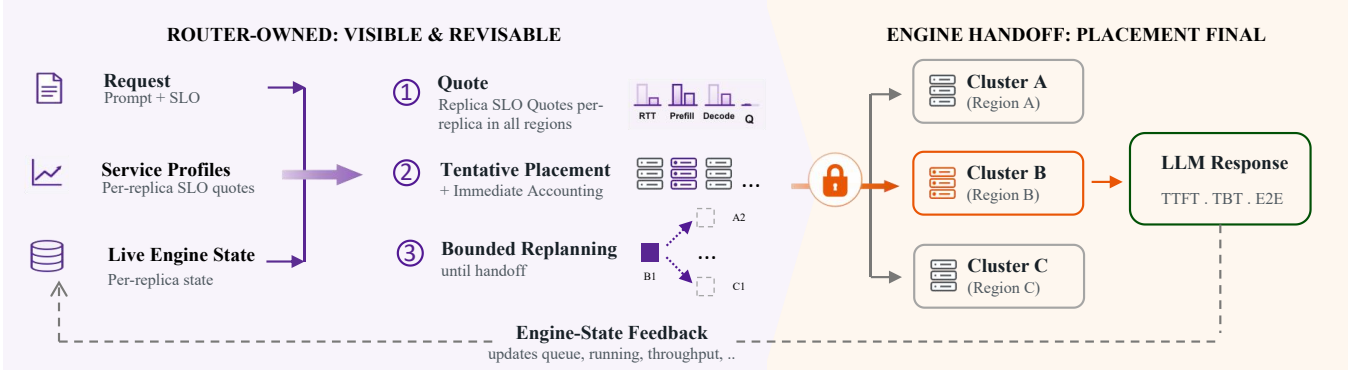


Figure 2: **RAVEL request lifecycle and ownership boundary.** RAVEL combines each request’s prompt and SLO with calibrated service profiles and live replica state to construct request-specific replica quotes. It then creates a virtual placement and immediately attributes the request’s prospective demand to its tentative replica. While the request remains Router-owned, subsequent arrivals may trigger bounded reassignment together with an atomic transfer of its prospective accounting. Capacity-gated dispatch initiates engine handoff; only a successful handoff makes placement final. Pending, running, and completed engine state continuously feeds back into later quotes and accounting updates.

where $m_{i,r}$ conservatively summarizes live and prospective Decode concurrency while logically deduplicating request identifiers. Feasibility therefore requires both bounds to hold: a lower predicted TTFT cannot compensate for a predicted TBT violation.

Completion-oriented requests instead require a prediction of total request-visible service time. RAVEL distinguishes expected output demand L_i^{exp} from a conservative risk estimate L_i^{risk} . Let $\hat{T}_{i,r}^{\text{risk}}(t)$ denote the predicted completion latency obtained from TTFT and the Decode service implied by L_i^{risk} . Completion feasibility is

$$\Phi_{i,r}^{\text{comp}}(t) = \mathbf{1} \left[\hat{T}_{i,r}^{\text{risk}}(t) \leq D_i^{\text{comp}} \right]. \quad (5)$$

The risk estimate gates feasibility, while the expected estimate contributes to shared service-pressure accounting. Separating the two avoids using an optimistic estimate to admit unsafe placements while also avoiding systematic over-accounting of prospective demand. Under sustained completion pressure, RAVEL may instead use a held-out, request-visible expected-output profile when constructing completion quotes; Section 3.5 describes this adaptation. Neither future arrivals nor the evaluated request’s realized output length is visible to the Router.

Appendix A.2 gives the complete quote equations, service-profile calibration procedure, runtime source of every quote term, output-demand fallbacks, and the exact construction of $\hat{T}_{i,r}^{\text{obj}}$ and $\mathbf{z}_{i,r}$.

3.3 Immediate and Transferable Demand Accounting

Revisability is safe only if tentative demand remains visible exactly once. RAVEL therefore associates every Router-

owned request with one tentative replica queue and one set of replica-dependent prospective charges. This makes the request visible immediately without making its placement final.

When request i receives a virtual placement at replica r , it enters r ’s Router-owned queue. Its prompt contributes to the queue-ahead Prefill work observed by later quotes, including quotes constructed later in the same planning pass. The planner additionally maintains lightweight signals for profiled completion-service pressure and LATENCY/TBT pressure. These prospective signals complement rather than replace live engine pending and running state.

If request i moves from replica r to r' , RAVEL transfers its queue ownership and recomputes all replica-dependent charges for the new target. For a virtual ledger Z with request-specific charge $z_{i,r}$, the transfer is conceptually

$$Z_r \leftarrow Z_r - z_{i,r}, \quad Z_{r'} \leftarrow Z_{r'} + z_{i,r}. \quad (6)$$

The source removal and destination addition occur atomically on the serialized control path. A planning snapshot therefore never observes the request at both replicas or at neither replica. The request joins the destination’s normal planning order rather than retaining its previous physical queue position.

Successful handoff ends Router queue ownership but does not erase the request’s effect on later decisions. Its active-service charge remains until terminal completion, while replica-reported pending and running state continues to inform subsequent quotes. Request identifiers are logically deduplicated wherever live and prospective state refer to the same request. Rejected handoff restores Router ownership and its prospective state; terminal completion releases the tracked charge and may trigger another planning pass.

Together, queue ownership and prospective accounting establish RAVEL’s central accounting invariant: *tentative work*

is immediately visible, but visibility does not make its placement final. Appendix A.3 specifies the exact charges, lifecycle operations, atomicity rules, and stale-callback protection.

3.4 Bounded Observation and Replanning

RAVEL replans only when newly observed information may change an uncommitted assignment, and it bounds both the movable set and the observation delay. A trigger starts a planning pass, eligibility determines which requests may move, and neither event changes the commitment boundary.

Only Router-owned requests are eligible for reassignment. Committed engine-owned work remains fixed but continues to affect later quotes. RAVEL also excludes requests with no alternative SLO-feasible target. Planning is event-driven rather than continuous: arrivals, mobility-hold expiration, relevant engine feedback, request termination, and rejected handoff may request a pass. Multiple requests for replanning are serialized and may be coalesced.

Slack-bounded mobility hold. For selected non-LATENCY requests, RAVEL preserves a short observation window before dispatch. The request is already virtually placed and accounted; the hold only keeps it Router-owned long enough for nearby arrivals to expose whether its tentative capacity assignment should change.

Let $\sigma_i^{\text{arr}} = \max\{0, \Delta_i - \min_r \hat{T}_{i,r}^{\text{obj}}(a_i)\}$ denote arrival-time planning slack, where Δ_i is the applicable TTFT or completion planning budget. For a hold-eligible request,

$$h_i = \begin{cases} \min\{\rho_i^{\min}, \sigma_i^{\text{arr}}\}, & \text{if } i \text{ is hold-eligible,} \\ 0, & \text{otherwise,} \end{cases} \quad (7)$$

where $\rho_i^{\min}$ is the minimum positive RTT to a remote serving cluster. Eligibility requires a causally detected RTT-scale burst, at least two SLO-feasible regions, and a non-LATENCY request. LATENCY requests always use $h_i = 0$. The RTT bound limits deliberate observation to a network timescale, while the slack bound prevents the hold from intentionally exceeding the request's predicted planning slack.

Hold expiration is not commitment. It makes a request dispatch-eligible, but the request remains revisable until successful handoff. This distinguishes RAVEL from fixed-delay late binding: held requests are already placed and accounted throughout the observation interval.

Bounded cohort. Each planning pass orders movable requests by

$$\kappa_i = (a_i + \Delta_i, a_i, id_i) \quad (8)$$

and ordinarily considers a prefix of at most K candidates. Requests outside the cohort retain their tentative owners and enter the planning snapshot as fixed context. Within the cohort, every selected placement immediately updates pass-local prospective state before the next candidate is evaluated.

Algorithm 1: RAVEL Router Control Loop

```

1 Function ONARRIVAL( $i$ ):
2    $Q_i \leftarrow \text{QUOTEREPLICAS}(i)$ 
3    $r_i \leftarrow \text{INITIALVIRTUALPLACEMENT}(i, Q_i)$ 
4    $\text{ASSIGNANDACCOUNT}(i, r_i)$ 
   // Demand becomes visible immediately
5    $h_i \leftarrow \text{MOBILITYHOLD}(i, Q_i)$ 
6   if  $h_i = 0$  or  $\text{COMPLETIONPRESSURE}()$  then
7      $\text{REQUESTREPLAN}()$ 
8   else
9      $\text{ARMREPLAN}(h_i)$ 

10 Function REPLAN():
11    $C \leftarrow \text{BOUNDEDMOVABLECOHORT}()$ 
   // Only Router-owned work is revisable
12    $X \leftarrow \text{SLOAWAREGREEDYPLACE}(C)$ 
13   for  $i \in C$  do
14      $\text{TRANSFERORREFRESH}(i, X_i)$ 

15 Function DISPATCH( $i$ ):
16   if  $\text{HELD}(i)$  or  $\text{FRONTIERBLOCKS}(i)$  then
17     return
18    $r \leftarrow \text{CURRENTTENTATIVEOWNER}(i)$ 
19    $\text{REMOVEFROMROUTERQUEUE}(i)$ 
20    $s \leftarrow \text{ATTEMPTHANDOFF}(i, r)$ 
21   if  $s = \text{REJECTED}$  then
22      $\text{RESTOREROUTEROWNERSHIP}(i, r)$ 
23   else if  $s = \text{COMMITTED}$  then
24      $\text{FINALIZEPLACEMENT}(i, r)$ 
   // Successful handoff makes placement final
25   else
26      $\text{SURFACEFAILUREAFTERCLEANUP}(i)$ 

```

Under completion pressure, the effective bound expands from K to the deployment's finite aggregate sequence capacity. With $R = |\mathcal{R}|$ candidate replicas and effective cohort bound K_{eff} , one greedy pass performs $O(K_{\text{eff}}R)$ request-replica quote comparisons. Completion pressure changes the amount of Router-owned state reconsidered, not the placement algorithm or ownership boundary.

Algorithm 1 summarizes the transitions that must preserve RAVEL's invariants: immediate arrival-time accounting, bounded reassignment, and failure-safe handoff.

Appendix A.4 gives the exact trigger predicates, eligibility filters, hold predicate, reassignment limits, full planning pseudocode, and configured complexity bounds.

3.5 Unified SLO-Aware Placement and Reversible Dispatch

Replica quotes identify which targets remain SLO-feasible; the placement policy determines how scarce feasible capacity is allocated among them. RAVEL uses one deterministic

lexicographic policy for both initial placement and bounded cohort replanning. Request classes affect the feasibility and objective fields exposed by a quote, but do not change the placement algorithm that consumes it.

Initial and cohort placement. On arrival, RAVEL constructs a quote for every candidate replica. If at least one replica is predicted feasible, RAVEL selects among feasible targets using capacity-pressure, efficiency, and stability signals. If no replica is predicted feasible, it first minimizes normalized predicted SLO violation and only then considers lower-priority efficiency signals. Unlike cache-aware routers that jointly optimize reusable KV state and load [40, 54], RAVEL treats prefix locality as a late tie-breaker: cache affinity cannot override request-level SLO feasibility.

Replanning applies the same placement policy to a bounded movable cohort. Requests are processed in the order defined by Equation 8. Each request is evaluated against the fixed system context together with the tentative assignments already selected earlier in the current pass. After selecting a target, RAVEL immediately updates the pass-local prospective state before evaluating the next request. Later requests therefore observe the demand induced by earlier tentative choices rather than reasoning from a stale pre-pass snapshot.

The placement key has three lexicographic tiers. First, *SLO safety* prioritizes predicted feasibility and, when no target is feasible, minimizes normalized predicted violation. Second, *capacity pressure* considers prospective sequence-capacity headroom, profiled completion-service pressure, and LATENCY/TBT pressure so that broadly substitutable work does not unnecessarily consume scarce SLO-feasible capacity. Third, *efficiency and stability* consider the request-specific predicted objective, marginal service cost, placement movement, prefix locality, and deterministic replica identity. Because these tiers are lexicographically ordered, a lower-priority load, locality, or stability benefit cannot compensate for an avoidable SLO miss. Appendix A.5 gives the exact implementation key, normalization rules, and tie-breaking direction.

Completion-pressure adaptation. Completion pressure changes the amount of Router-owned work reconsidered and the completion-demand estimate supplied to the common planner; it does not introduce a second placement or dispatch policy. Using only causally observed arrivals and serving state, RAVEL detects sustained workflow-completion pressure and expands the movable cohort from the ordinary bound K to the deployment’s finite aggregate sequence capacity. When a held-out semantic profile indexed only by request-visible workflow metadata is available, RAVEL may also replace the ordinary conservative output-demand estimate with the profile’s expected-output estimate when constructing completion quotes. These adaptations change planner inputs and the amount of reconsidered state, not the placement algorithm itself. Neither exposes future arrivals or the evaluated request’s

realized generation length, and all requests continue through the same planner and reversible dispatch path. Appendix A.6 gives the exact pressure predicate, entry and exit conditions, cohort expansion, and profile fallbacks.

Shallow reversible dispatch frontier. Placement revisability is useful only while work remains under Router ownership. RAVEL therefore limits how much work may cross from Router-owned state into engine-facing state at once. The dispatch frontier exposes only enough Prefill work to cover one calibrated engine quantum and a bounded control horizon, subject to current engine headroom. Excess work remains in Router-owned queues, where it continues to contribute to prospective accounting and remains eligible for replanning.

The frontier is a capacity-gated dispatch boundary rather than a second admission or predicted-miss withholding policy. A dispatch-eligible request at the frontier proceeds whenever its tentative target has sufficient engine headroom. Thus, the frontier bounds the amount of work approaching commitment while allowing eligible work to progress whenever capacity is available. It preserves Router-level visibility and revisability without creating a protected cohort or a separate placement policy. Appendix A.7 specifies the exact frontier construction, headroom test, wake-up events, and concurrent-handoff bound.

Engine-local TBT protection. Replica-level feasibility is meaningful only if local execution preserves the service assumptions used to construct the corresponding quote. Prior phase-aware serving systems mitigate Prefill–Decode interference through chunking, disaggregation, or related execution controls [3, 13, 32, 59]. RAVEL uses a narrower engine-local mechanism to preserve its predicted TBT conditions rather than introducing another geographic scheduling decision.

When Chunked Prefill is enabled and LATENCY Decode is active, RAVEL exports the tightest applicable TBT requirement together with deadline metadata to the local serving adapter. The adapter combines this requirement with a conservative calibrated Prefill envelope to bound the next Prefill quantum. LATENCY requests receive deadline/TBT-aware local priority, whereas non-LATENCY work retains stable FCFS behavior.

These controls enforce the execution assumptions underlying replica quotes; they do not alter geographic placement or create a second Router-level scheduler. The Router remains solely responsible for regional placement and pre-handoff reassignment. Appendix A.8 gives the conditional activation rule, chunk-bound equation, priority encoding, and scheduler wrapper.

3.6 Commitment and Engine Handoff

Commitment is an ownership transition rather than a timer or queue event. Only successful engine handoff transfers a

request from Router-owned, revisable state to engine-owned, final state. Tentative placement, mobility-hold expiration, dispatch eligibility, and temporary removal from a Router queue are all non-commitment events.

For a dispatch-eligible request, RAVEL reads its current tentative owner, removes it from that owner’s Router queue, and asks the target replica to register the request. Registration alone does not commit placement. If the target rejects registration, RAVEL restores Router ownership and prospective state at the tentative replica, after which the request again follows the ordinary dispatch and replanning rules.

After registration succeeds, RAVEL creates and tracks the corresponding engine-facing posting task. Successful registration together with successful posting-task creation constitutes *successful handoff*. Only then does the virtual placement become final. If posting-task creation fails before this boundary, RAVEL aborts the partial registration, cleans attempt-specific state, and surfaces the failure without reporting a committed placement.

Failures after successful handoff belong to the committed attempt. RAVEL performs terminal abort and cleanup and refreshes its planning state, but does not reinterpret a post-handoff failure as another placement opportunity. Attempt identifiers prevent delayed callbacks from an earlier attempt from modifying ownership or accounting associated with a newer attempt.

Committed work remains visible to later planning. Replica pending and running state contributes to later quotes, and its tracked active-service charge persists until terminal completion. Releasing that charge may request another planning pass so that newly available capacity can affect requests that remain Router-owned.

RAVEL therefore separates three lifecycle events: *arrival makes demand visible*; *Router ownership keeps placement revisable*; *successful engine handoff makes placement final*. Appendix A.9 gives the complete ownership state machine, failure transitions, cleanup rules, and attempt-identifier semantics.

4 Implementation

We implement RAVEL in Python as an asynchronous control layer in front of geo-distributed vLLM serving replicas [15]. Our implementation targets vLLM 0.8.5.post1 and its frozen V0 scheduler contract. All control-plane operations that modify Router-owned queues, perform replanning, or initiate dispatch are serialized by a single `asyncio` scheduler lock. New arrivals, mobility-hold expirations, engine feedback, and explicit replanning requests therefore enter the same serialized scheduling path, preventing concurrent passes from racing on tentative ownership or prospective accounting. Terminal callbacks may asynchronously release tracked service charges, but any resulting replanning again enters this serialized path.

Serving profiles. RAVEL derives serving estimates from externally calibrated schema-v2 profiles rather than hard-coded token rates. Each profile carries an engine fingerprint and is bound to one serving cluster; formal runs require a valid external profile for every cluster. At runtime, RAVEL linearly interpolates Prefill and Decode measurements between calibrated concurrency points and clamps queries outside the measured range to the nearest endpoint. The engine adapter additionally constructs a conservative Prefill envelope from the maximum calibrated slope and intercept across concurrency levels. For multi-stage completion requests, the Router may also load an offline semantic output profile indexed only by request-visible workflow metadata, such as (*stage_id*, *stage_num*). Neither profile exposes future arrivals or realized output length.

vLLM integration and local TBT control. RAVEL integrates with vLLM without modifying its source tree. At process startup, a lightweight adapter feature-detects and wraps `Scheduler._get_priority` and `Scheduler._schedule_chunked_prefill`. If either required hook is unavailable, startup fails rather than silently disabling RAVEL’s local controls. For LATENCY requests, the adapter encodes a Router-derived deadline-order key together with the applicable TBT requirement in a reserved priority range; non-LATENCY requests in the mixed path retain ordinary FCFS priority. When LATENCY Decode is active, the adapter combines the tightest active TBT requirement with the conservative Prefill envelope to derive a block-aligned bound on the next Prefill quantum. It temporarily reduces `max_num_batched_tokens` for that Chunked-Prefill scheduling iteration and restores the configured value in a `finally` block. The bound therefore affects only the current local scheduling decision and does not introduce a separate Router-level placement policy.

Asynchronous handoff and failure handling. Engine handoff is implemented as an explicit asynchronous ownership transition. After Router-side admission and budget checks succeed, the target replica first registers the request, after which RAVEL creates and tracks the corresponding engine-facing posting task. Successful replica registration together with successful posting-task creation constitutes the handoff defined in Section 3.6; only then does placement become final.

If replica registration is rejected, the request is restored to its Router-owned queue together with its prospective state and remains eligible for the ordinary mobility and replanning rules. Failures while creating or executing an engine-facing posting task are surfaced to the control plane rather than silently converted into new Router-level placements. Posting attempts carry attempt identifiers so that delayed callbacks from an older attempt cannot modify ownership or accounting associated with a newer one.

Table 2: Common experimental configuration.

Component	Configuration
Serving topology	5 replicas in 3 clusters (2/2/1)
Model and hardware	Qwen3-1.7B; one RTX 4090 per replica
Serving engine	vLLM 0.8.5.post1 V0; bfloat16
Maximum model length	16,384 tokens
Engine capacity	64 sequences; 16,384 batched tokens
KV-cache configura- tion	Block size 16; automatic prefix caching
Directed WAN RTT	10.1–37.9 ms
Requests per main cell	500; seed 42
Offered-load factors	1, 2, 4, 8, 12, 16×

After successful handoff, replica-reported pending and running state continues to inform subsequent Router quotes. The corresponding active-service charge remains visible until terminal completion. At request termination, RAVEL releases that charge and requests another serialized scheduling pass so that newly available capacity can affect requests that remain Router-owned.

5 Evaluation

We evaluate RAVEL along five dimensions. **RQ1:** Does RAVEL improve request-level SLO attainment as offered load increases? **RQ2:** Do the gains persist across repeated runs? **RQ3:** Which RAVEL mechanisms produce the end-to-end gains? **RQ4:** Are replica-specific quotes accurate enough to support feasibility decisions? **RQ5:** Is online replanning fast enough for the serving control path?

The main matrix reports one 500-request replay per policy–workload–load cell. A separate five-run campaign evaluates replay-to-replay variation at 16× load. These two result contexts are reported separately and are not averaged together.

5.1 Experimental Setup

Testbed. All policies run on the same five-replica physical-WAN testbed. The replicas are distributed across three geographic clusters in a 2/2/1 configuration. Each replica serves Qwen3-1.7B on one NVIDIA RTX 4090 using vLLM 0.8.5.post1’s V0 scheduler and bfloat16 weights. The directed physical WAN paths have measured RTTs ranging from 10.1 to 37.9 ms. The remaining common engine and replay settings are summarized in Table 2.

Workloads and SLOs. We use three workloads that expose different sources of placement uncertainty. *LMSYS* represents heterogeneous conversational traffic. *Burst* preserves the request contents of its source trace but replaces the arrival timestamps with a bursty schedule, stressing decisions made

from transient queue state. *DeepResearch* represents multi-stage completion workflows whose output demand depends on request-visible stage metadata.

Each request carries an SLO class and a request-specific multiplier. The base budgets are 0.8 s for TTFT, 80 ms for TBT, and 8 s for completion latency, after which the configured multiplier is applied. A LATENCY request succeeds only if it satisfies both its TTFT bound and every TBT bound. THROUGHPUT and COLLECTIVE requests succeed if their end-to-end completion latency remains within their completion budget.

Replay protocol. We replay each workload in open loop. At load factor x , every original inter-arrival interval is divided by x while request order, contents, output metadata, and SLOs are preserved. Before each cell, we reset replica prefix caches and start a fresh policy run. All policies receive the same request-visible information and are evaluated under the same model, topology, WAN paths, and replay schedule. The main matrix contains one 500-request replay per cell. The repeated-run campaign is conducted separately using five paired replays at 16× load.

Policies and metrics. We compare RAVEL with RoundRobin (RR), LeastLoad (LL), RD, DualMap, and SkyWalker. For the single-run headline results, the reported numerical gains use DualMap as the comparison policy. The repeated-run figure follows the designated best-baseline protocol used by that campaign.

Our primary metric is request-level SLO attainment. Secondary metrics are time to first token (TTFT), end-to-end latency (E2E), and time per output token (TPOT). TTFT and E2E are measured from arrival at the Router, including time spent under Router ownership as well as WAN and engine delay.

5.2 End-to-End Performance

RAVEL’s advantage grows with contention, when a placement selected from arrival-time state is more likely to become stale before engine handoff. As shown in Figure 3, RAVEL maintains the highest request-level SLO attainment across the evaluated load sweep.

At 16× load, RAVEL improves SLO attainment over DualMap by 8.8 percentage points on LMSYS, 11.2 percentage points on Burst, and 13.4 percentage points on DeepResearch. The corresponding P95 TTFT reductions range from 49.1% to 80.5%, while P95 E2E reductions range from 15.5% to 56.8%.

These gains are consistent with RAVEL retaining the ability to revise a tentative placement while the request remains Router-owned. As load increases, queueing and service estimates change more rapidly, making an irrevocable decision based only on arrival-time state increasingly costly. RAVEL

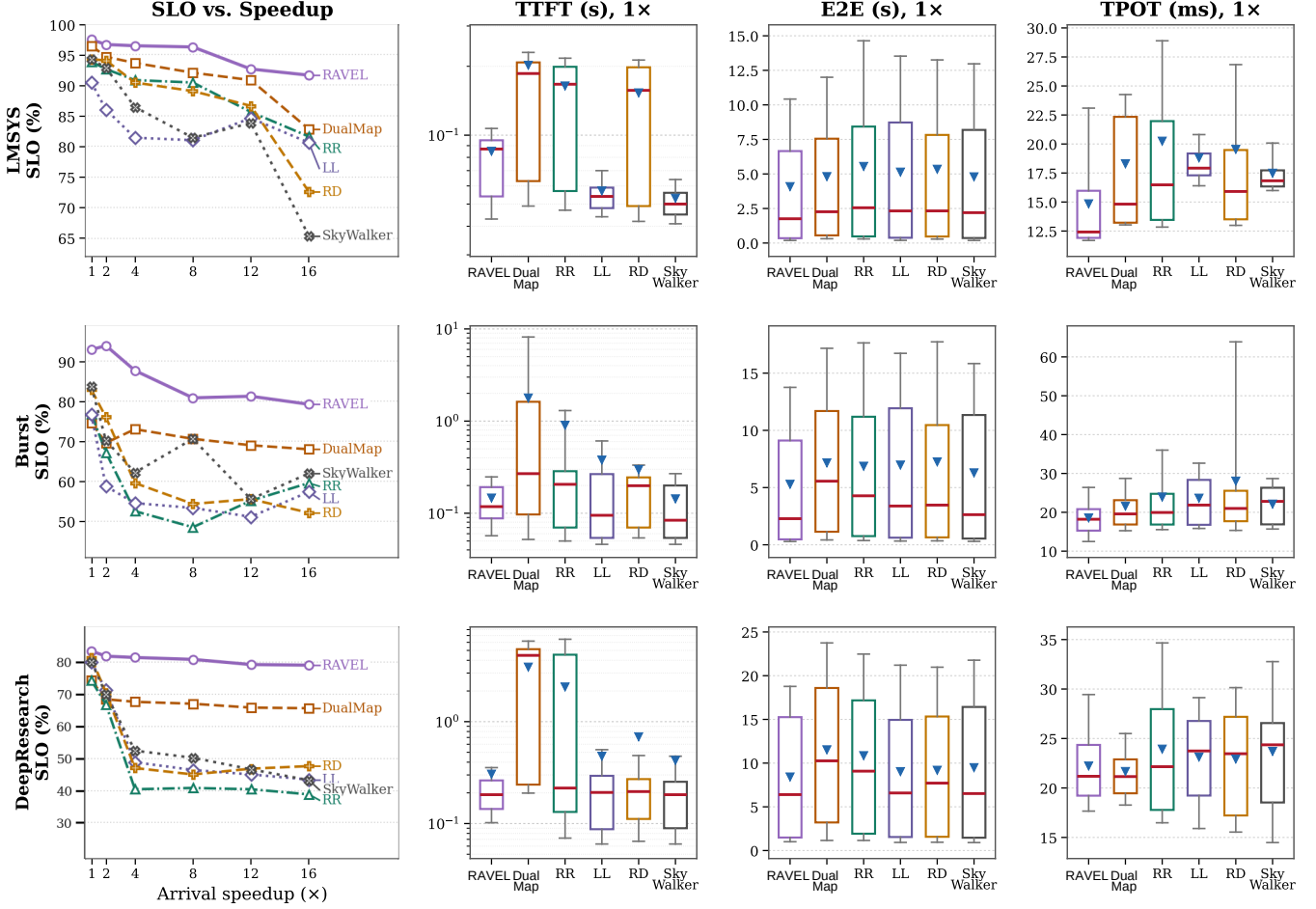


Figure 3: End-to-end results for LMSYS (top), Burst (middle), and DeepResearch (bottom), using one 500-request replay per policy–workload–load cell. The first column reports request-level SLO attainment as offered load increases from $1\times$ to $16\times$; the remaining columns report per-request TTFT, E2E, and TPOT distributions at $1\times$. In each box plot, the red line marks the median, the box spans the 25th–75th percentiles, the whiskers show the 10th–90th percentiles, and the inverted triangle marks the mean. RAVEL maintains the highest SLO attainment as contention grows and outperforms the best baseline on all three workloads.

can respond to these changes until handoff while prospective demand accounting remains active from the moment of arrival.

The $1\times$ distributions in Figure 3 show why raw latency alone is not sufficient to characterize this objective. The policies’ latency distributions overlap for several metrics, yet RAVEL can still attain more request-level SLOs. It selectively uses available slack: a request with sufficient slack may tolerate a small additional delay if doing so preserves a scarce SLO-feasible replica for a more region-constrained request. Thus, the relevant objective is not to minimize every individual latency sample, but to allocate heterogeneous regional capacity so that more requests satisfy their own SLOs.

5.3 Repeated-Run Robustness

The single-run matrix establishes behavior across workloads and offered loads, but one replay per cell cannot quantify replay-to-replay variation. We therefore conduct a separate five-run campaign at $16\times$ load. For each workload, every run contains a complete 500-request replay, and the RAVEL result is paired with the result from the campaign’s designated best baseline under the same run conditions. Each paired run contributes one difference in request-level SLO attainment; requests within a replay are not treated as independent statistical samples.

The repeated runs preserve RAVEL’s advantage on all three workloads. The mean gains are 4.1 percentage points on LMSYS, 7.5 percentage points on Burst, and 17.4 percentage points on DeepResearch. For each workload, the 95% confidence interval over the five paired run-level differences lies

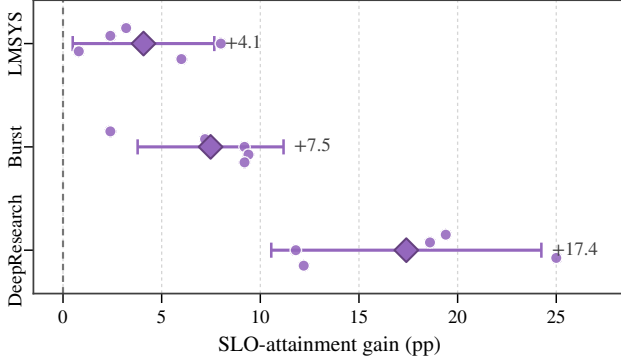


Figure 4: **Repeated-run robustness at $16\times$ load.** Each small circle reports the paired RAVEL–baseline difference in request-level SLO attainment for one complete 500-request replay. The large diamond marks the mean over five paired runs, and the horizontal whisker shows the corresponding 95% confidence interval computed from the five run-level differences. The plotted comparison uses the designated best baseline for the corresponding workload. Mean gains are 4.1, 7.5, and 17.4 percentage points for LMSYS, Burst, and DeepResearch, respectively.

above zero. This supports the conclusion that the average improvement is reproducible under replay variation, while not implying that every individual request or every possible run must improve.

The five-run values are separate from the single-run matrix in Figure 3. They use a different experimental batch and statistical unit, so the 17.4-percentage-point DeepResearch value should not be substituted for the 13.4-percentage-point single-run result. The individual paired values and statistical construction are reported in Appendix B.

5.4 Mechanism Contributions

Both pre-handoff revisability and engine-local TBT protection contribute to RAVEL’s end-to-end SLO gains. We isolate these mechanisms on Burst at $12\times$, where burst-driven queue changes are pronounced but the system has not collapsed into universal overload.

We compare full RAVEL with two variants. *FixedPlacement* retains RAVEL’s SLO-aware initial placement, prospective accounting, and engine-local TBT guard, but disables reassignment after the initial choice. It therefore isolates the value of keeping placement revisable. *NoTBTGuard* retains RAVEL’s global routing, prospective accounting, and replanning, but disables the engine-local Prefill control used to protect active LATENCY Decode requests.

Full RAVEL attains 81.2% request-level SLO attainment. FixedPlacement attains 68.0%, a reduction of 13.2 percentage points, while NoTBTGuard attains 71.4%, a reduction of 9.8 percentage points. Thus, neither a strong initial placement nor global replanning without local TBT protection is sufficient.

The component-level metrics explain these losses. FixedPlacement obtains a lower P95 TTFT than RAVEL (2.490 versus 2.905 s), but its LATENCY-request TBT-violation rate increases from 6.6% to 37.6%. Its P95 E2E also rises to 21.347 s, compared with 18.985 s for RAVEL. An irrevocable geographic choice can therefore create downstream Decode pressure that the remaining local mechanism cannot fully repair.

NoTBTGuard reduces P95 TTFT further to 1.951 s and has nearly the same P95 E2E as RAVEL (18.958 versus 18.985 s), yet its TBT-violation rate reaches 42.0%. Optimizing aggregate TTFT or E2E is therefore insufficient when individual requests also carry inter-token deadlines.

Because FixedPlacement retains prospective accounting, this experiment specifically measures the contribution of placement revisability while holding that accounting mechanism fixed. It should not be interpreted as an independent ablation of prospective accounting itself.

5.5 Quote Accuracy

RAVEL’s quotes are useful only if they preserve the feasibility decision for each request, rather than merely achieving a small average prediction error. In the workload- $16\times$ analysis, the empirical false-feasible and false-infeasible rates are 6.2% and 1.0%, respectively.

For a selected final-handoff record i , we define normalized quote error as

$$e_i = \frac{\hat{L}_i - L_i}{B_i},$$

where $\hat{L}_i$ is the predicted latency, L_i is the observed latency, and B_i is the corresponding request-specific SLO budget. For LATENCY requests, L_i denotes TTFT; for completion-oriented requests, L_i denotes E2E. Negative error is optimistic, whereas positive error is conservative.

Figure 6 shows that most normalized errors remain near zero, although completion quotes exhibit a longer optimistic tail. More importantly, the quotes usually preserve the decision-relevant feasibility classification. Of the 1,672 records quoted as feasible, 104 later violate the corresponding SLO, giving a 6.2% empirical false-feasible rate. Of the 828 records quoted as infeasible, only 8 complete feasibly, giving a 1.0% empirical false-infeasible rate.

These measurements evaluate the placements RAVEL actually selected and handed off; they do not observe counterfactual execution on unselected replicas. The reported rates therefore characterize the empirical quality of the online decision signal, rather than providing a formal guarantee that every feasible quote will meet its SLO.

5.6 Planner Overhead

RAVEL’s replanning cost is small on the actual online path, while a controlled benchmark shows how planner latency

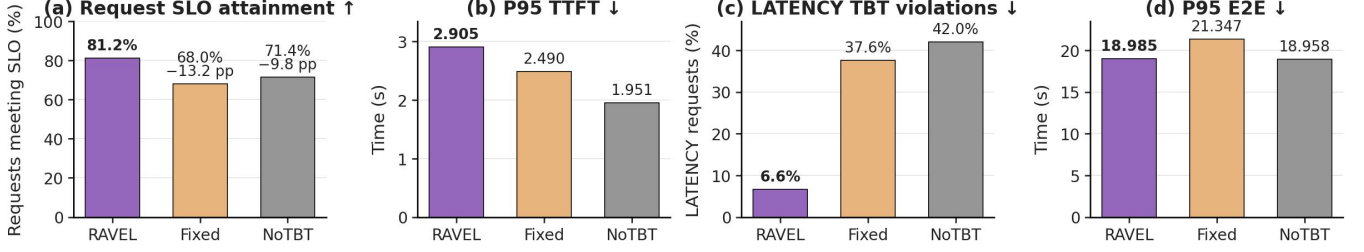


Figure 5: Mechanism ablation on the 500-request Burst workload at $12\times$ load. FixedPlacement retains RAVEL’s initial SLO-aware placement, prospective accounting, and TBT guard but disables pre-handoff reassignment. NoTBTGuard retains global RAVEL routing and replanning but disables the engine-local TBT guard. The panels report (a) request-level SLO attainment, (b) P95 TTFT, (c) the TBT-violation rate among LATENCY requests, and (d) P95 E2E. Higher is better only for SLO attainment.

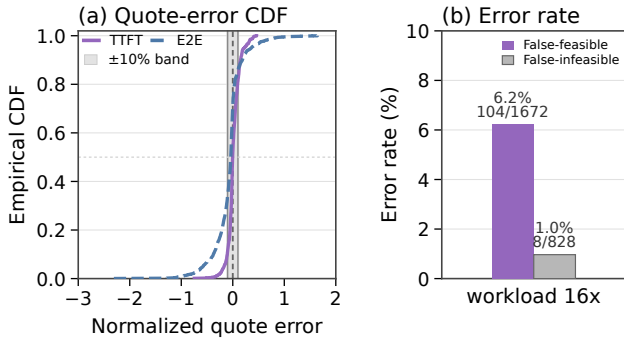


Figure 6: **Quote accuracy in the workload- $16\times$ analysis.** Panel (a) shows the empirical CDF of normalized quote error $(\hat{L}_i - L_i)/B_i$ for LATENCY (TTFT) and completion (E2E) requests. The shaded band denotes errors within $\pm 10\%$ of the applicable SLO budget. Panel (b) reports feasibility-classification error rates over the same $N = 2,500$ selected final-handoff records. Among 1,672 quoted-feasible decisions, 104 are false feasible (6.2%); among 828 quoted-infeasible decisions, 8 are false infeasible (1.0%).

scales with the bounded cohort size. We denote by K the candidate cohort bound for one replanning pass; the normal online configuration caps it at $K = 64$.

On the actual Burst $12\times$ execution path, a scheduling pass takes 0.998 ms at the median and 2.101 ms at P95. The mean observed cohort contains 3.65 requests, its P95 is 7, and its maximum is 9.

The controlled benchmark measures the cost of approaching the configured bound. At $K = 64$, median planner latency is 10.99 ms for mixed cohorts and 20.10 ms for completion-oriented cohorts. Increasing the cohort to the stress-only value $K = 128$ raises these values to 23.47 and 42.75 ms, respectively. The shaded ranges in Figure 7 are measurement dispersion across repetitions, not confidence intervals.

The gap between the controlled bound and the observed runtime cohorts explains why actual passes are much faster than the worst-case benchmark. These measurements characterize the evaluated five-replica deployment; extrapolating

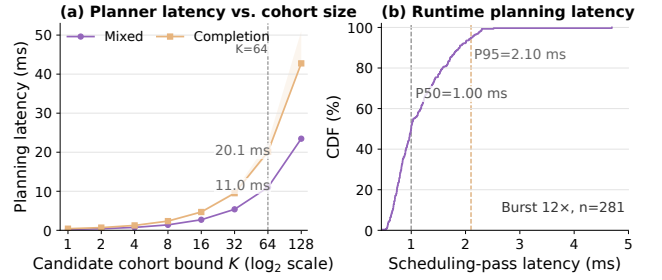


Figure 7: **Planner overhead on the five-replica testbed.** Panel (a) reports median planner latency as the candidate cohort bound K increases on a log₂ scale. The shaded bands span P5–P95 over 50 measured repetitions per point after five warm-up repetitions; the dashed line marks the configured online bound $K = 64$, while $K = 128$ is a stress-only point. Panel (b) shows the empirical CDF of scheduling-pass latency for 281 actual passes from Burst at $12\times$ load.

the same latency to substantially larger replica sets requires a separate fleet-size study.

Evaluation takeaway. Together, the experiments show that RAVEL’s gains persist across workloads and repeated runs, arise from the combination of pre-handoff revisability and engine-local protection, and are supported by decision-useful quotes and millisecond-scale online planning. The evidence characterizes the evaluated five-replica deployment and does not constitute a formal SLO guarantee for arbitrary fleet sizes.

6 Related Work

LLM serving engines and phase scheduling. Modern LLM serving systems improve execution efficiency through iteration-level batching and KV-cache management [15, 53], scalable and parallel inference [4, 19, 33, 46], optimized attention and speculative execution [6, 29], and structured or adapter-multiplexed serving [38, 58]. A complementary line of work explicitly manages the distinct resource demands

of Prefill and Decode. Sarathi-Serve uses Chunked Prefill to reduce Prefill–Decode interference [3], DistServe disaggregates the two phases and provisions them against latency objectives [59], and Splitwise and ShuffleInfer similarly separate or coordinate phase execution to reduce interference [13, 32]. These systems primarily improve how admitted work executes. RAVEL operates at a different control layer: it determines geographic placement and preserves bounded placement revisability before engine handoff, while relying on the local serving engine to execute admitted requests.

Distributed and cache-aware LLM routing. Distributed serving systems increasingly route requests using runtime load, reusable KV state, and adaptive scheduling signals. Preble jointly considers reusable KV state and computation load [40], while DualMap balances cache affinity against load and performs hotspot-aware rebalancing [54]. Other work explores learned or adaptive request scheduling [9, 20, 56]. A related body of work improves KV-cache reuse, storage, and transfer through cache-centric serving and prefix-aware execution [1, 10, 16, 34, 50–52]. These techniques improve destination selection or reduce the cost of executing a request at a selected destination. RAVEL does not introduce a new cache primitive or routing score. Instead, it separates destination selection from commitment: an arrived request contributes prospective demand to its tentative replica immediately, while that replica may still change before successful engine handoff.

Geo-distributed and network-aware placement. Recent systems explicitly incorporate geographic and network heterogeneity into LLM placement. SkyWalker balances cross-region load while preserving KV-cache locality [49]; GORGON incorporates WAN latency, queueing state, and Prefill/cache cost into cross-region decisions [44]; and Solyx uses serving telemetry together with WAN measurements for per-request cross-site placement [5]. Helix further considers heterogeneous GPU and network resources in LLM serving [27], while network-aware ML schedulers such as CASSINI coordinate placement with communication conditions [35]. More broadly, private-WAN systems such as SWAN and B4 demonstrate the importance of heterogeneous inter-datacenter paths and traffic constraints [12, 14]. These systems sharpen the decision of *where* work should execute. RAVEL makes an additional control decision first-class: *when should that placement become irreversible?* It answers this question by retaining Router-owned tentative placement and prospective accounting until the engine-handoff boundary.

SLO- and application-aware LLM scheduling. Prior systems adapt LLM execution and routing to request-specific latency, deadline, or quality objectives. JITServe schedules heterogeneous request classes from imprecise request infor-

mation [57], QUARTZ targets TTFT SLOs through quantile-aware routing and queueing [25], Andes considers quality-of-experience for streamed generation [23], and AdaServe customizes speculative decoding to per-request SLO requirements [18]. Libra combines request partitioning with SLO-aware scheduling under dynamic load [36], while complementary work studies serving fairness, SLO metrics, and serving-performance characterization [2, 39, 45]. Application-aware systems exploit structure across related LLM calls: Parrot coordinates semantic dependencies across requests [21], Agentix schedules agent programs as first-class execution entities [26], and Ayo optimizes end-to-end LLM applications [43]. Agent frameworks and benchmarks further expose increasingly multi-stage and dependency-rich workloads [24, 28, 48, 60]. RAVEL applies request-specific SLO semantics at a different control point: it uses WAN delay and replica serving state to determine which geographic capacity remains feasible before handoff, while its engine-local controls preserve the execution assumptions used by those placement decisions.

Reassignment, migration, and online commitment. Serving systems can preserve flexibility after an initial assignment through runtime migration, state movement, preemption, or dynamic orchestration [8, 30, 41, 42, 47]. Llumnix, for example, dynamically reschedules requests across model instances and uses live migration to improve load balancing and isolation [42]. These mechanisms retain placement flexibility after engine-managed execution state has materialized. More broadly, delay scheduling shows that waiting can expose better placement opportunities [55], while deadline-aware and online scheduling systems study finite slack, incomplete information, and scalable decision making [11, 17, 22, 31, 37]. RAVEL chooses an earlier commitment boundary. A request may be reassigned while it remains Router-owned, but its placement becomes final after successful engine handoff, before changing placement would require moving engine-managed request state. Pre-handoff replanning and runtime migration are therefore complementary: the former can avoid some state movement, while the latter can correct imbalance that emerges after execution begins.

7 Conclusion

Geo-distributed LLM serving must route requests online even though regional capacity is request-dependent: a region that is interchangeable for one request may be essential for satisfying another request’s SLO. Later arrivals can reveal this opportunity cost, but waiting for such information consumes the same requests’ SLO slack. RAVEL addresses this tension by separating demand visibility from placement commitment. It accounts each request immediately at a tentative replica, while keeping that placement revisable as long as

the request remains Router-owned and committing it only at successful engine handoff. Across LMSYS, bursty traffic, and multi-stage DeepResearch workloads, RAVEL consistently improves request-level SLO attainment while maintaining competitive latency, with particularly large gains under several high-contention settings. More broadly, RAVEL shows that an online serving system need not postpone demand accounting to preserve the flexibility to exploit information revealed by later arrivals.

References

- [1] AGARWAL, S., SUNDARESAN, S., MITRA, S., MAHAPATRA, D., GUPTA, A., SHARMA, R., KAPU, N. J., YU, T., AND SAINI, S. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [2] AGRAWAL, A., KEDIA, N., MOHAN, J., PANWAR, A., KWATRA, N., GULAVANI, B. S., RAMJEE, R., AND TUMANOV, A. VIDUR: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems* (2024), vol. 6, pp. 351–366.
- [3] AGRAWAL, A., KEDIA, N., PANWAR, A., MOHAN, J., KWATRA, N., GULAVANI, B., TUMANOV, A., AND RAMJEE, R. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 117–134.
- [4] AMINABADI, R. Y., RAJBHANDARI, S., AWAN, A. A., LI, C., LI, D., ZHENG, E., RUWASE, O., SMITH, S., ZHANG, M., RASLEY, J., AND HE, Y. Deepspeed- inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis* (2022), IEEE, pp. 1–15.
- [5] BERNHARD, A., AND KATLA, N. Solix AI grid: Hardware-telemetry-aware routing across geographically distributed GPU clusters, 2026. arXiv:2606.15050v1.
- [6] DAO, T. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.
- [7] DEAN, J., AND BARROSO, L. A. The tail at scale. *Communications of the ACM* 56, 2 (2013), 74–80.
- [8] FU, Y., XUE, L., HUANG, Y., BRABETE, A.-O., USTIUGOV, D., PATEL, Y., AND MAI, L. ServerlessLLM: Low-Latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 135–153.
- [9] FU, Y., ZHU, S., SU, R., QIAO, A., STOICA, I., AND ZHANG, H. Efficient llm scheduling by learning to rank. In *Advances in Neural Information Processing Systems 37* (2024), Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 59006–59029.
- [10] GAO, B., HE, Z., SHARMA, P., KANG, Q., JEVDJIC, D., DENG, J., YANG, X., YU, Z., AND ZUO, P. Cost-Efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 111–126.
- [11] GOG, I., SCHWARZKOPF, M., GLEAVE, A., WATSON, R. N. M., AND HAND, S. Firmament: Fast, centralized cluster scheduling at scale. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (Savannah, GA, Nov. 2016), USENIX Association, pp. 99–115.
- [12] HONG, C.-Y., KANDULA, S., MAHAJAN, R., ZHANG, M., GILL, V., NANDURI, M., AND WATTENHOFER, R. Achieving high utilization with software-driven wan. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 15–26.
- [13] HU, C., HUANG, H., XU, L., CHEN, X., XU, J., CHEN, S., FENG, H., WANG, C., WANG, S., BAO, Y., SUN, N., AND SHAN, Y. Inference without interference: Disaggregate llm inference for mixed downstream workloads, 2024.
- [14] JAIN, S., KUMAR, A., MANDAL, S., ONG, J., POUTIEVSKI, L., SINGH, A., VENKATA, S., WANDERER, J., ZHOU, J., ZHU, M., ZOLLA, J., HÖLZLE, U., STUART, S., AND VAHDAT, A. B4: Experience with a globally-deployed software defined wan. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 3–14.
- [15] KWON, W., LI, Z., ZHUANG, S., SHENG, Y., ZHENG, L., YU, C. H., GONZALEZ, J. E., ZHANG, H., AND STOICA, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles* (2023), SOSP ’23, ACM, pp. 611–626.
- [16] LEE, W., LEE, J., SEO, J., AND SIM, J. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 155–172.
- [17] LI, W., HE, X., LIU, Y., LI, K., CHEN, K., GE, Z., GUAN, Z., QI, H., ZHANG, S., AND LIU, G. Flow scheduling with imprecise knowledge. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (Santa Clara, CA, Apr. 2024), USENIX Association, pp. 95–111.
- [18] LI, Z., CHEN, Z., DELACOURT, R., OLIARO, G., WANG, Z., CHEN, Q., LIN, S., YANG, A., ZHANG, Z., CHEN, Z., LAI, Y.-H., CHENG, X., MIAO, X., AND JIA, Z. AdaServe: Accelerating multi-SLO LLM serving with SLO-customized speculative decoding. In *Proceedings of the 21st European Conference on Computer Systems* (2026), EuroSys ’26, ACM, pp. 36–54.
- [19] LI, Z., ZHENG, L., ZHONG, Y., LIU, V., SHENG, Y., JIN, X., HUANG, Y., CHEN, Z., ZHANG, H., GONZALEZ, J. E., AND STOICA, I. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)* (Boston, MA, July 2023), USENIX Association, pp. 663–679.
- [20] LIM, G., ZHAO, W., GODFREY, B., SHAN, J., XU, L., AND XIE, L. Lodestar: An online-learning llm inference router, 2026.
- [21] LIN, C., HAN, Z., ZHANG, C., YANG, Y., YANG, F., CHEN, C., AND QIU, L. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 929–945.
- [22] LIU, C. L., AND LAYLAND, J. W. Scheduling algorithms for multi-programming in a hard-real-time environment. *Journal of the ACM* 20, 1 (1973), 46–61.
- [23] LIU, J., CHUNG, J.-W., WU, Z., LAI, F., LEE, M., AND CHOWDHURY, M. Andes: Defining and enhancing quality-of-experience in llm-based text streaming services, 2024.
- [24] LIU, X., YU, H., ZHANG, H., XU, Y., LEI, X., LAI, H., GU, Y., DING, H., MEN, K., YANG, K., ZHANG, S., DENG, X., ZENG, A., DU, Z., ZHANG, C., SHEN, S., ZHANG, T., SU, Y., SUN, H., HUANG, M., DONG, Y., AND TANG, J. Agentbench: Evaluating llms as agents, 2025.
- [25] LIU, Z., ZHENG, Y., KONG, F., AND ZHAO, Z. QUARTZ: Quantile-aware routing and queueing for TTFT SLOs in LLM serving. In *Findings of the Association for Computational Linguistics: ACL 2026* (San Diego, California, United States, July 2026), Association for Computational Linguistics, pp. 37886–37896.
- [26] LUO, M., SHI, X., CAI, C., ZHANG, T., WONG, J., WANG, Y., WANG, C., HUANG, Y., CHEN, Z., GONZALEZ, J. E., AND STOICA, I. Agentix: An efficient serving engine for LLM agents as general programs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)* (Renton, WA, May 2026), USENIX Association, pp. 2443–2459.

- [27] MEI, Y., ZHUANG, Y., MIAO, X., YANG, J., JIA, Z., AND VINAYAK, R. Helix: Serving large language models over heterogeneous gpus and network via max-flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (2025), ACM, pp. 586–602.
- [28] MIALON, G., FOURRIER, C., SWIFT, C., WOLF, T., LECUN, Y., AND SCIALOM, T. Gaia: a benchmark for general ai assistants, 2023.
- [29] MIAO, X., OLIARO, G., ZHANG, Z., CHENG, X., WANG, Z., ZHANG, Z., WONG, R. Y. Y., ZHU, A., YANG, L., SHI, X., SHI, C., CHEN, Z., ARFEEN, D., ABHYANKAR, R., AND JIA, Z. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (2024), ACM, pp. 932–949.
- [30] MIAO, X., SHI, C., DUAN, J., XI, X., LIN, D., CUI, B., AND JIA, Z. Spotservice: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2024), ACM, pp. 1112–1127.
- [31] OUSTERHOUT, K., WENDELL, P., ZAHARIA, M., AND STOICA, I. Sparrow: Distributed, low latency scheduling. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* (2013), SOSP '13, ACM, pp. 69–84.
- [32] PATEL, P., CHOUKSE, E., ZHANG, C., SHAH, A., ÍÑIGO GOIRI, MALEKI, S., AND BIANCHINI, R. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)* (2024), IEEE, pp. 118–132.
- [33] POPE, R., DOUGLAS, S., CHOWDHURY, A., DEVLIN, J., BRADBURY, J., LEVSKAYA, A., HEEK, J., XIAO, K., AGRAWAL, S., AND DEAN, J. Efficiently scaling transformer inference, 2022.
- [34] QIN, R., LI, Z., HE, W., CUI, J., REN, F., ZHANG, M., WU, Y., ZHENG, W., AND XU, X. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)* (Santa Clara, CA, Feb. 2025), USENIX Association, pp. 155–170.
- [35] RAJASEKARAN, S., GHOBADI, M., AND AKELLA, A. CASSINI: Network-Aware job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (Santa Clara, CA, Apr. 2024), USENIX Association, pp. 1403–1420.
- [36] RUAN, C., CHEN, Y., TIAN, D., SHI, Y., WU, Y., LI, J., AND LI, C. Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)* (Renton, WA, May 2026), USENIX Association, pp. 1243–1258.
- [37] SCHWARZKOPF, M., KONWINSKI, A., ABD-EL-MALEK, M., AND WILKES, J. Omega: Flexible, scalable schedulers for large compute clusters. In *Proceedings of the 8th ACM European Conference on Computer Systems* (2013), ACM, pp. 351–364.
- [38] SHENG, Y., CAO, S., LI, D., HOOPER, C., LEE, N., YANG, S., CHOU, C., ZHU, B., ZHENG, L., KEUTZER, K., GONZALEZ, J. E., AND STOICA, I. S-LoRA: Serving thousands of concurrent LoRA adapters. In *Proceedings of Machine Learning and Systems* (2024), vol. 6.
- [39] SHENG, Y., CAO, S., LI, D., ZHU, B., LI, Z., ZHUO, D., GONZALEZ, J. E., AND STOICA, I. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 965–988.
- [40] SRIVATSA, V., HE, Z., ABHYANKAR, R., LI, D., AND ZHANG, Y. Preble: Efficient distributed prompt scheduling for LLM serving. In *The Thirteenth International Conference on Learning Representations* (2025).
- [41] STRATI, F., MCALLISTER, S., PHANISHAYEE, A., TARNAWSKI, J., AND KLIMOVIC, A. Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving, 2024.
- [42] SUN, B., HUANG, Z., ZHAO, H., XIAO, W., ZHANG, X., LI, Y., AND LIN, W. Lumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 173–191.
- [43] TAN, X., JIANG, Y., YANG, Y., AND XU, H. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2025), ACM, pp. 1302–1316.
- [44] TONIOLO, A. R., THORSTENSON, R., AND DINESH, A. GORGO: Online tuning for cross-region network-aware LLM serving, 2026. arXiv:2602.11688v3.
- [45] WANG, Z., LI, S., ZHOU, Y., LI, X., ZHANG, Z., CAM-TU, N., GU, R., TIAN, C., CHEN, G., AND ZHONG, S. Revisiting service level objectives and system level metrics in large language model serving, 2024.
- [46] WU, B., LIU, S., ZHONG, Y., SUN, P., LIU, X., AND JIN, X. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (2024), ACM, pp. 640–654.
- [47] WU, B., ZHU, R., ZHANG, Z., SUN, P., LIU, X., AND JIN, X. dLoRA: Dynamically orchestrating requests and adapters for LoRA LLM serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 911–927.
- [48] WU, Q., BANSAL, G., ZHANG, J., WU, Y., LI, B., ZHU, E., JIANG, L., ZHANG, X., ZHANG, S., LIU, J., AWADALLAH, A. H., WHITE, R. W., BURGER, D., AND WANG, C. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023.
- [49] XIA, T., MAO, Z., KERNEY, J., JACKSON, E. J., LI, Z., XING, J., SHENKER, S., AND STOICA, I. SkyWalker: A locality-aware cross-region load balancer for LLM inference. In *Proceedings of the 21st European Conference on Computer Systems* (2026), EuroSys '26, ACM, pp. 1349–1364.
- [50] YANG, J., HOU, B., WEI, W., BAO, Y., AND CHANG, S. Kvlunk: Accelerating large language models via efficient kv cache reuse. In *Advances in Neural Information Processing Systems 38* (2025), Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 148312–148339.
- [51] YAO, J., LI, H., LIU, Y., RAY, S., CHENG, Y., ZHANG, Q., DU, K., LU, S., AND JIANG, J. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems* (2025), ACM, pp. 94–109.
- [52] YE, L., TAO, Z., HUANG, Y., AND LI, Y. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2024), Association for Computational Linguistics, pp. 11608–11620.
- [53] YU, G.-I., JEONG, J. S., KIM, G.-W., KIM, S., AND CHUN, B.-G. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)* (Carlsbad, CA, July 2022), USENIX Association, pp. 521–538.
- [54] YUAN, Y., ZUO, P., WANG, B., CHEN, Z., TAN, Z., AND YU, Z. DualMap: Enabling both cache affinity and load balancing for distributed LLM serving. In *The Fourteenth International Conference on Learning Representations* (2026).

- [55] ZAHARIA, M., BORTHAKUR, D., SARMA, J. S., ELMELEEGY, K., SHENKER, S., AND STOICA, I. Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European Conference on Computer Systems* (2010), EuroSys '10, ACM, pp. 265–278.
- [56] ZHANG, D., HAN, J., ZHANG, K., WEI, X., SHEN, S., FANG, C., YU, W., ZHOU, J., AND CHEN, R. Simple is better: Multiplication may be all you need for llm request scheduling, 2026.
- [57] ZHANG, W., WU, Z., MU, Y., NING, R., LIU, B., SARDA, N., LEE, M., AND LAI, F. JITServe: SLO-aware LLM serving with imprecise request information. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)* (Renton, WA, May 2026), USENIX Association, pp. 825–848.
- [58] ZHENG, L., YIN, L., XIE, Z., SUN, C., HUANG, J., YU, C., CAO, S., KOZYRAKIS, C., STOICA, I., GONZALEZ, J., BARRETT, C., AND SHENG, Y. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37* (2024), Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 62557–62583.
- [59] ZHONG, Y., LIU, S., CHEN, J., HU, J., ZHU, Y., LIU, X., JIN, X., AND ZHANG, H. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 193–210.
- [60] ZHOU, S., XU, F. F., ZHU, H., ZHOU, X., LO, R., SRIDHAR, A., CHENG, X., OU, T., BISK, Y., FRIED, D., ALON, U., AND NEUBIG, G. Webarena: A realistic web environment for building autonomous agents, 2024.

A Detailed Design and Operational Semantics

A.1 Common Request State and Notation

RAVEL separates *placement ownership* from engine execution state. Before successful engine handoff, a request is Router-owned: it has one tentative replica, contributes prospective demand to that replica, and may be replanned subject to the mobility rules below. Successful engine handoff transfers ownership to the serving engine and makes the geographic placement final. Engine-side execution may subsequently move through Prefill, Decode, completion, or failure states, but these events do not reopen geographic placement.

For request i , we use the following notation: a_i is its Router arrival time; o_i is its client region; $r_i(t)$ is its tentative replica while Router-owned; v_i is the current ownership/version counter used by versioned transfers; and h_i identifies the current engine attempt. Attempt identifiers isolate asynchronous completion, cancellation, and cleanup events across retries.

The transport path distinguishes three internal states: WAITING_MOVABLE, COMMITTING, and PREFILL_LOCKED. A dispatch claim moves a request from WAITING_MOVABLE to the transient COMMITTING state; this is not a second placement decision. If target registration fails, Router ownership is restored. Placement becomes final only after the target accepts the request and the engine-facing posting task is created and tracked successfully. RAVEL does not perform ordinary geographic rebinds after this successful handoff.

Across these states, RAVEL enforces five invariants.

1. **Single ownership.** Every nonterminal request has exactly one control-plane or engine owner.
2. **Single tentative placement.** Every Router-owned request is attributed to exactly one tentative replica at a time.
3. **Exactly-once accounting.** A transferable prospective charge is removed from its old owner before being installed at its new owner; a request identifier is logically deduplicated when Router, pending, and running sets overlap.
4. **Attempt isolation.** A callback from attempt h cannot clean state belonging to a newer attempt $h' > h$.
5. **No post-commit rebind.** Ordinary RAVEL replanning applies only before successful handoff; post-handoff imbalance is left to the engine or to a separate migration mechanism.

A.2 Replica SLO Quote Construction

A placement decision consumes a causal quote

$$Q_{i,r}(t) = \left(\Phi_{i,r}(t), \hat{T}_{i,r}^{\text{obj}}(t), \mathbf{z}_{i,r}(t) \right), \quad (9)$$

where $\Phi_{i,r}$ is predicted SLO feasibility, $\hat{T}_{i,r}^{\text{obj}}$ is the class-specific ordering objective, and $\mathbf{z}_{i,r}$ contains secondary capacity-pressure and stability signals. Quotes are predictions constructed only from information available at decision time; they are not formal SLO guarantees. Their inputs come from request-visible SLO metadata, directed WAN measurements, calibrated per-cluster service profiles, currently observed engine state, Router-owned queue state, and tentative assignments already selected in the same planning pass. Shadow prefix locality is used only as a late tie-breaker, and the evaluated request's realized output length is never used by its quote.

Calibrated service curves. For cluster c and active Decode concurrency n , the schema-v2 service profile stores an affine Prefill model

$$P_{c,n}(p) = b_{c,n} + a_{c,n}p \quad (10)$$

and a measured post-first-token Decode time per token $d_c(n)$. Profiles are calibrated independently for each serving class and contain an engine fingerprint. At runtime RAVEL linearly interpolates between measured concurrency points and clamps values outside the calibrated concurrency range to the nearest endpoint rather than extrapolating without bound. Because $P_{c,n}$ is measured while n Decode sequences are active, RAVEL does not add a separate ad-hoc Decode-interference tax to the Prefill estimate.

Queue-ahead Prefill expansion. Let p_i be the full prompt-token charge of request i . Without engine-certified cached-token residency, the Router's shadow prefix estimate is used only as a locality tie-breaker and does not reduce p_i . For candidate replica r , let E_r denote observed engine-pending Prefill tokens, $B_{i,r}$ the prompt tokens in Router-owned requests ordered no later than i , N_r^{eng} the number of engine-pending requests, and $N_{i,r}^q$ the corresponding number of Router-owned queue-ahead requests. With n_r currently running Decode sequences, RAVEL estimates

$$\hat{S}_{i,r}^{\text{pre}} = a_{c(r),n_r}(E_r + B_{i,r} + p_i) + b_{c(r),n_r}(N_r^{\text{eng}} + N_{i,r}^q + 1). \quad (11)$$

During cohort replanning, $B_{i,r}$ and $N_{i,r}^q$ additionally include tentative assignments chosen earlier in the same planning pass. Thus a later candidate observes the prospective work induced by earlier choices rather than a stale snapshot.

The point TTFT estimate is

$$\widehat{\text{TTFT}}_{i,r}^{\text{point}}(t) = (t - a_i) + \rho_{o_i,c(r)} + \hat{S}_{i,r}^{\text{pre}}, \quad (12)$$

where $\rho_{o_i,c(r)}$ is the directed client-to-cluster WAN delay. The elapsed term explicitly charges Router observation time against the request's remaining SLO budget.

Prospective Decode occupancy. Let O_r be the union of request identifiers visible in the Router-owned queue, engine-pending set, and engine-running set for replica r . RAVEL forms a prospective sequence count

$$m_{i,r} = \max(1, |O_r \cup \{i\}|, N_r^q + N_r^{\text{eng}} + N_r^{\text{run}} + \mathbf{1}[i \notin O_r]), \quad (13)$$

and queries the calibrated Decode curve at $m_{i,r}$, with profile-level clamping at the measured endpoint. The set term prevents the same request from being charged twice when lifecycle views overlap.

LATENCY feasibility. For a LATENCY request with TTFT and TBT limits D_i^{TTFT} and D_i^{TBT} ,

$$\Phi_{i,r}^{\text{lat}}(t) = \mathbf{1}[\widehat{\text{TTFT}}_{i,r}(t) \leq D_i^{\text{TTFT}} \wedge d_{c(r)}(m_{i,r}) \leq D_i^{\text{TBT}}]. \quad (14)$$

The two constraints are not combined into a weighted score: a low TTFT cannot compensate for a predicted TBT violation.

Completion feasibility and output-demand fallback. For completion-oriented requests, RAVEL distinguishes request-visible expected output demand L_i^{exp} from conservative risk demand L_i^{risk} :

$$\widehat{T}_{i,r}^{\text{point}} = \widehat{\text{TTFT}}_{i,r}^{\text{point}} + \max(0, L_i^{\text{exp}} - 1)d_{c(r)}(m_{i,r}), \quad (15)$$

$$\widehat{T}_{i,r}^{\text{risk}} = \widehat{\text{TTFT}}_{i,r}^{\text{point}} + \max(0, L_i^{\text{risk}} - 1)d_{c(r)}(m_{i,r}). \quad (16)$$

Ordinary completion feasibility uses

$$\Phi_{i,r}^{\text{comp}}(t) = \mathbf{1}[\widehat{T}_{i,r}^{\text{risk}}(t) \leq D_i^{\text{comp}}], \quad (17)$$

while the expected estimate is used to represent shared service pressure. For multi-stage COLLECTIVE requests, we use a held-out semantic profile indexed by $(\text{stage_id}, \text{stage_num})$. Missing contexts fall back first to a stage-level aggregate and then to the declared routing hint. The profile is constructed from training data only; the evaluated request’s realized generation length is never an input to its placement quote.

Residual diagnostics. RAVEL records type-specific objective residuals in a bounded history. For n available residuals and risk level ϵ , the empirical finite-sample rank is

$$k = \lceil (n+1)(1-\epsilon) \rceil. \quad (18)$$

If $k > n$, the quote is marked uncalibrated. In our evaluation, residuals are recorded for diagnostics but do not alter placement decisions. The reported results therefore do not rely on a cold-start residual guarantee.

The quote procedure is:

QUOTE_{REPLICA}(i, r, t): (1) read calibrated service curves and currently observed engine state; (2) charge full prompt work and EDF-ordered Router work ahead of i ; (3) add tentative work already selected earlier in the current pass; (4) compute Eq. 12; (5) form the request-ID-deduplicated prospective Decode count in Eq. 13; (6) evaluate the LATENCY or completion feasibility predicate; and (7) return feasibility, the class-specific objective, and secondary pressure/locality signals.

A.3 Prospective Accounting Lifecycle

RAVEL uses prospective state for two related purposes: to make newly arrived work visible before commitment and to prevent a planning pass from repeatedly assigning work against an unchanged snapshot.

Queue and service debt. The queue-ahead terms in Eq. 11 are derived from current Router-owned and engine-visible state. In addition, the WorkYield planner maintains virtual admission ledgers that make repeated placement decisions account for already admitted work even when engine lifecycle feedback has not yet arrived.

For a LATENCY request,

$$u_{i,r} = \frac{d_{c(r)}(1)}{D_i^{\text{TBT}}}, \quad A_r = \sum_{j: \text{latency owner}(j)=r} u_{j,r}. \quad (19)$$

Both numerator and denominator are seconds, so $u_{i,r}$ is dimensionless and represents one-token-period service demand. This ledger is a placement balancing signal rather than a reconstruction of the instantaneous vLLM queue.

For all requests for which service accounting is active, define

$$s_{i,r}^{\text{own}} = \widehat{s}_{i,r}^{\text{own,pre}} + \widehat{s}_{i,r}^{\text{own,dec}}, \quad L_r = \sum_{j: \text{service owner}(j)=r} s_{j,r}^{\text{own}}. \quad (20)$$

The Prefill component contains only the request’s own profiled service (including the profile intercept), not queue-ahead debt; the Decode component is the request-visible expected service used by the corresponding quote. Thus L_r is measured in seconds of profiled service.

Exactly-once registration and transfer. Each persistent virtual charge is accompanied by an owner field on the request. Registration is idempotent: if the owner field already exists, the same request is not added again. During cohort replanning, RAVEL first removes the mobile candidates’ old charges from the fixed planning base. It then processes candidates in deadline order; after each tentative target is chosen, the request’s charge is added to the pass-local target ledger immediately. After the cohort is complete, persistent ownership

is transferred by subtracting the old charge once and adding the newly computed charge once. This realizes deferred commitment without deferred accounting.

For completion-oriented accounting, RAVEL explicitly tracks requests carrying a service virtual owner and removes their active service charge on the matching terminal callback before requesting another scheduling pass. The LATENCY token-period ledger is used as a cumulative placement-balancing signal and is therefore not interpreted as an exact count of currently running Decode requests; actual Decode feasibility continues to use the request-ID-deduplicated live prospective sequence count in Eq. 13.

Operationally, arrival installs the request at one tentative owner; a replan transfers that ownership and its prospective charge exactly once; failed handoff restores Router ownership; successful handoff makes the placement final while keeping pending/running engine state visible; and terminal completion or failure releases the matching active service charge.

The accounting transfer sequence is:

TRANSFERORREFRESH($i, r \rightarrow r'$): (1) verify that i is still Router-owned and that its attempt/version is current; (2) remove i 's old virtual charges from the planning base; (3) recompute $Q_{i,r'}$ against fixed and pass-local state; (4) if r' is selected, update the pass-local work/count/service ledgers immediately; (5) atomically replace the request's persistent owner/charge fields; and (6) update the recorded tentative quote. A failed version/attempt check leaves the current owner unchanged.

A.4 Bounded Observation and Replanning

RAVEL replans from causally observed state rather than polling for an unbounded future window. Scheduling operations are serialized by a single asynchronous scheduler lock. New arrivals, hold expiration, terminal feedback, explicit control-plane replan requests, and completion-pressure changes may request another pass; repeated requests for a pass can be coalesced before the next serialized execution.

Slack-bounded observation. The mobility hold is enabled only for selected non-LATENCY requests. Let δ_i be the most recently observed interarrival gap and let $\mathcal{D}_i$ contain positive RTTs from i 's client region to remote clusters. The burst predicate is

$$\text{Burst}_i = \mathbf{1}[\mathcal{D}_i \neq \emptyset \wedge \delta_i \leq \max \mathcal{D}_i]. \quad (21)$$

A hold is considered only if the request is predicted feasible in at least two clusters. Let

$$\text{slack}_i = D_i - \min_r \hat{T}_{i,r}^{\text{obj}}. \quad (22)$$

RAVEL uses the smallest positive remote RTT as the observation scale and bounds it by available slack:

$$\Delta_i = \begin{cases} \min(\min \mathcal{D}_i, \text{slack}_i), & \text{if } i \text{ is non-LATENCY, } \text{Burst}_i = 1, \text{ and} \\ & i \text{ has at least two feasible clusters,} \\ 0, & \text{otherwise.} \end{cases} \quad (23)$$

LATENCY requests are not deliberately held. They may participate in pre-handoff reassignment only when a remote SLO-feasible target exists. The hold changes dispatch eligibility, not accounting: the request is tentatively placed and visible throughout $[a_i, a_i + \Delta_i]$.

Movable cohort. The planner orders movable requests by

$$\kappa_i = (a_i + D_i, a_i, id_i), \quad (24)$$

where D_i denotes the applicable TTFT limit for LATENCY requests and completion limit otherwise. A request is excluded if it is no longer Router-owned, has crossed the handoff boundary, or has exhausted the one-rebind limit. The ordinary candidate bound is derived from serving sequence capacity and is set to `max_num_seqs`. Under completion pressure, Section A.6, the bound expands to aggregate sequence capacity across replicas.

Fixed and pass-local context. A planning pass freezes the noncandidate engine and Router context. Fixed Router requests that precede candidate i under Eq. 24 contribute their full prompt work and request intercept to i 's quote. After a target is selected for one mobile candidate, its prompt work, request count, service charge, and applicable TBT charge are immediately added to pass-local state before the next candidate is evaluated.

REPLAN: (1) collect the bounded movable cohort and sort it by Eq. 24; (2) freeze current noncandidate engine/Router state; (3) remove candidates' old transferable charges from the fixed planning base; (4) for each candidate in order, quote every legal replica, choose the minimum lexicographic placement key, and update pass-local state immediately; (5) apply only still-current assignments, respecting attempt/version and one-rebind guards; and (6) dispatch newly eligible work through the ordinary frontier.

Let N be the movable cohort size, R the number of replicas, and F the number of fixed Router-owned requests scanned to build queue-ahead prefixes. The WorkYield planner has planning cost

$$O(N \log N + F \log F + NR + F), \quad (25)$$

plus constant-size quote interpolation. This bound is independent of generated token count; RAVEL does not send one Router scheduling event per generated token.

A.5 Deterministic Placement Key

RAVEL avoids combining unlike SLO and load quantities in a fitted weighted sum. The WorkYield planner compares candidate replica r for request i using a lexicographic key whose conceptual order is

$$K_{i,r} = (\neg\Phi_{i,r}, O_{i,r}, \ell_{i,r}^{\text{obj}}, \ell_{i,r}^{\text{TTFT}}, A_r + u_{i,r}, L_r + s_{i,r}^{\text{own}}, \widehat{T}_{i,r}^{\text{obj}}/D_i, M_{i,r}, \text{moved}_{i,r}, id_r), \quad (26)$$

with lower values preferred. Terms that are inapplicable to a request class are set to their neutral value rather than introducing a class-specific placement algorithm. The ordering is intentional: feasibility and normalized SLO violation dominate capacity pressure; capacity pressure dominates predicted objective and marginal service; movement, prefix locality, and stable replica identity are used only as late tie-breakers.

The normalized lateness terms are

$$\ell_{i,r}^{\text{obj}} = \frac{\max(0, \widehat{T}_{i,r}^{\text{obj}} - D_i)}{\max(D_i, \epsilon)}, \quad (27)$$

$$\ell_{i,r}^{\text{TTFT}} = \frac{\max(0, \widehat{\text{TTFT}}_{i,r} - D_i^{\text{TTFT}})}{\max(D_i^{\text{TTFT}}, \epsilon)}. \quad (28)$$

$O_{i,r}$ is the pending-capacity overflow cost when that guard is applicable. The convex marginal-service term is

$$M_{i,r} = \frac{(W_r + s_{i,r}^{\text{own}})^2 - W_r^2}{D_i^2}, \quad (29)$$

where W_r is the profiled service already represented at r in the current planning state. Equation 29 is the discrete increment of a quadratic congestion potential, not a learned weight.

Initial single-request placement uses the same safety ordering. After feasibility, normalized violation, and virtual-load terms tie, it may consult current pressure and shadow prefix locality before the stable replica ID. Prefix locality never precedes feasibility or normalized SLO violation.

Two-request example. Suppose request 1 is feasible in A and B , while request 2 is feasible only in A . When request 1 is tentatively assigned during a cohort pass, RAVEL immediately adds its prompt/service charge to the selected region's pass-local state. Request 2 is therefore quoted against that tentative assignment. If placing request 1 in A makes request 2 infeasible while placing request 1 in B preserves both requests' feasible assignments, the first component of Eq. 26 dominates later locality or movement preferences. This is the concrete mechanism by which request-dependent regional substitutability affects placement without requiring a fitted cross-metric score.

A.6 Completion-Pressure Adaptation

The completion-pressure path is enabled by request-visible workflow semantics, not by a dataset name. A request is con-

sidered a workflow-completion request when it is COLLECTIVE and has more than one stage. RAVEL tracks the count, first and latest arrival times, and minimum observed completion SLO among such arrivals. It also records whether any nonworkflow traffic has appeared.

Let C_{seq} be aggregate sequence capacity across replicas and let D_{min} be the minimum completion SLO among the observed workflow arrivals. After a minimum number of samples,

$$q_{\text{obs}} = \frac{N_{\text{arr}} - 1}{t_{\text{last}} - t_{\text{first}}}, \quad q_{\text{act}} = \frac{C_{\text{seq}}}{\max(1\text{ s}, 0.5D_{\text{min}})}. \quad (30)$$

Arrival pressure is observed once $q_{\text{obs}} \geq q_{\text{act}}$. RAVEL requires

$$N_{\text{arr}} \geq \max\left(4, \min\left(16, \left\lfloor \frac{C_{\text{seq, replica}}}{4} \right\rfloor\right)\right) \quad (31)$$

before evaluating the rate predicate, where the per-replica quantity is the configured per-replica pending/sequence limit. Effective completion pressure additionally requires that workflow-completion traffic has been observed, no nonworkflow request has been observed, and outstanding work is nonzero. There is no separately tuned numerical hysteresis band: the observed-arrival-pressure bit is causal and sticky within the run, while the effective pressure path becomes inactive when the eligibility conditions no longer hold and the active flag is cleared when outstanding work drains to zero.

Completion pressure changes only two planner inputs.

1. The movable-candidate ceiling expands from the ordinary bound K to aggregate sequence capacity across the deployment.
2. If the held-out semantic output profile covers the request context, completion quotes use its expected output demand rather than the ordinary conservative upper estimate for the placement-time expected workload.

Completion pressure does not introduce a second Router-side admission queue or withhold requests predicted to miss their SLOs. All candidates remain in the common WorkYield placement path and are dispatched through the CenterYield frontier.

Semantic output profile. The held-out schema-v2 semantic profile indexes ($stage_id, stage_num$). For covered context x it stores the training mean and a finite-sample $q_{0.95}$ value. Lookup follows

$$x = (stage_id, stage_num) \rightarrow stage_id \text{ aggregate} \rightarrow \text{declared routing hint} \quad (32)$$

Neither the current request's realized output length nor future arrivals enter this lookup.

When an active completion request terminates, Unified removes its tracked service virtual charge, deletes the request’s service-owner metadata, and requests another serialized scheduling pass. Consequently, newly available completion capacity can influence requests that remain Router-owned.

A.7 Shallow Reversible Dispatch Frontier

The CenterYield frontier limits how much Router-visible prompt work can cross toward engine state at once. Let

$$H = \max_{c \neq c'} p_{c,c'} \quad (33)$$

be the largest configured directed inter-cluster control RTT, and let B_{iter} be the engine’s `max_num_batched_tokens`. For replica r , let $a_{c(r),0}$ be the calibrated idle/fastest Prefill seconds per token. RAVEL sets the prompt frontier

$$F_r = B_{\text{iter}} + \left\lceil \frac{H}{a_{c(r),0}} \right\rceil. \quad (34)$$

Thus the engine can hold approximately one complete scheduling quantum plus the prompt work that the fastest calibrated Prefill path could consume during one worst-case control RTT.

At dispatch time, if E_r is currently observed engine-pending prompt work, the frontier headroom is

$$G_r = \max(0, F_r - E_r). \quad (35)$$

The effective dispatch budget is the minimum of G_r and the replica’s ordinary engine budget. A Router-owned request may cross the frontier only if its full conservative prompt charge fits this budget. Eligible requests are ordered by their applicable absolute SLO deadline, then arrival time and request ID.

In our five-replica deployment, the maximum directed RTT is $H = 37.9$ ms. We use $B_{\text{iter}} = 16384$ tokens, `max_num_seqs`=64, and 16-token blocks. The second term of Eq. 34 is deployment-specific and is recomputed from the calibrated service profile.

The frontier does not withhold predicted misses in a protected queue. If an eligible request fits the capacity budget, it is dispatched. If target registration rejects the request, the request is restored to Router-owned state and remains visible to the ordinary replan/dispatch path. Work that does not yet fit stays Router-owned, continues to contribute to prospective planning, and remains movable until successful handoff.

A.8 Engine-Local TBT Protection

RAVEL’s engine adapter protects the local execution assumptions used by LATENCY quotes. The adapter is installed only when the target vLLM exposes the expected scheduler hooks; it fails fast if `Scheduler._get_priority` or

`Scheduler._schedule_chunked_prefill` is absent rather than silently running an unvalidated compatibility path.

Let

$$T = \min_{i \in \mathcal{D}_r^{\text{lat}}} D_i^{\text{TBT}} \quad (36)$$

be the tightest active TBT among LATENCY Decode requests at replica r . From the bound profile, the adapter takes

$$a_r^{\text{guard}} = \max_n a_{c(r),n}, \quad b_r^{\text{guard}} = \max_n b_{c(r),n}. \quad (37)$$

For block size g and ordinary token limit B_{max} , it computes

$$q_{\text{raw}} = \left\lfloor \frac{\max(0, T - b_r^{\text{guard}})}{a_r^{\text{guard}}} \right\rfloor, \quad (38)$$

$$q_{\text{blk}} = \min \left(B_{\text{max}}, \max \left(g, g \left\lfloor \frac{q_{\text{raw}}}{g} \right\rfloor \right) \right). \quad (39)$$

If n_r^{dec} Decode sequences are active, the temporary block-aligned batch budget is

$$B_r^{\text{guard}} = g \left\lceil \frac{n_r^{\text{dec}} + q_{\text{blk}}}{g} \right\rceil. \quad (40)$$

The adapter sets

$$B_{\text{iter}} \leftarrow \min(B_{\text{max}}, B_r^{\text{guard}}) \quad (41)$$

for that Chunked-Prefill scheduling call only and restores B_{max} in a finally block.

If no active Decode carries a LATENCY TBT obligation, the adapter follows the ordinary vLLM scheduling path. If even one minimum-size block exceeds the profiled TBT budget, the adapter must still provide vLLM a legal block and therefore cannot claim a TBT guarantee in that physical regime; the Router’s replica-feasibility test is intended to avoid such placements, and any realized violation remains in the evaluation denominator.

Phase-aware priority metadata carries a remaining deadline budget rather than a Router monotonic-clock timestamp. The engine reconstructs a local deadline from its own arrival clock, avoiding an implicit cross-host monotonic-clock synchronization assumption. Non-RAVEL priorities occupy a different range, so the adapter remains inert for ordinary vLLM work.

A.9 Commitment and Engine-Handoff State Machine

RAVEL defines commitment as a successful ownership transition, not as a timer expiration or queue removal. The Router may choose a tentative target, hold a request, replan it, and mark it dispatch-eligible without making its placement final.

The engine-facing handoff path is: (1) freeze the latest Router quote for observability; (2) ask the target replica to register the request; (3) create the asynchronous engine-facing

HTTP posting task; and (4) install that task in the attempt-indexed task table. The quote-freeze callback is diagnostic and occurs before registration; it does not itself commit placement. Successful registration together with successful task creation/tracking is the paper-level commitment boundary. Before that point, a dispatch claim may either succeed or roll back: target registration rejection restores the Router-owned request, and setup failure cleans the matching partial attempt. After successful handoff, completion, failure, and cancellation perform attempt-specific cleanup but do not reopen geographic placement. Callbacks whose attempt identifier no longer matches the active attempt are ignored.

Task cleanup is attempt-aware: a deletion request carrying attempt h is ignored when the stored task belongs to a different attempt. The same attempt identifier is passed through completion/failure callbacks, preventing a delayed callback from an earlier posting attempt from releasing state belonging to a newer one.

Before successful handoff, RAVEL changes only Router/transport metadata and prospective accounting; the serving engine has not yet established the engine-managed execution state whose migration would be required after execution starts. RAVEL therefore obtains pre-handoff placement flexibility without KV-state migration. Once handoff succeeds, later pending/running engine state remains visible to subsequent quotes, but the committed request itself is no longer an ordinary geographic-replanning candidate.

A request remains reassignable while it is Router-owned. Its placement becomes final only after the target replica has accepted registration and the corresponding engine-facing posting task has been created and tracked successfully.

B Repeated-Run Robustness

This appendix reports a five-run paired robustness study at $16\times$ load. The repeated-run statistics are reported separately from single-run headline results because they summarize different experimental repetitions.

B.1 Result Context and Comparator Definitions

For DeepResearch at $16\times$ load, the single-run comparison in the main evaluation reports 79.0% request-level SLO attainment for RAVEL and 65.6% for DualMap, a gain of 13.4 percentage points. The repeated-run study uses DualMap as the fixed comparator for all three workloads and all five repetitions.

The five-run DeepResearch study yields a mean paired gain of 17.40 percentage points over DualMap. This value is reported separately from the 13.4 percentage-point single-run

Table 3: Mean paired SLO-attainment gains at $16\times$ load across five repetitions. Intervals are two-sided 95% Student- t CIs over paired gains.

Workload	Mean (pp)	95% CI (pp)
LMSYS	4.08	[0.49, 7.67]
Burst	7.48	[3.78, 11.18]
DeepResearch	17.40	[10.56, 24.24]

result because the two statistics summarize different experimental repetitions.

B.2 Five-Run Experimental Protocol

For each workload $w \in \{\text{LMSYS}, \text{Burst}, \text{DeepResearch}\}$ at $16\times$ load, the robustness experiment consists of five complete paired replays. Within paired repetition j , RAVEL and the workload’s fixed comparator use the same request set and order, trace timing, random seed, serving endpoints, physical-WAN topology, model/engine configuration, cache reset procedure, and warm-up protocol.

For workload w and paired run j , define the SLO-attainment gain

$$\Delta_{w,j} = \text{SLOAttain}_{\text{RAVEL},w,j} - \text{SLOAttain}_{\text{DualMap},w,j}. \quad (42)$$

The five-run paired mean is

$$\bar{\Delta}_w = \frac{1}{5} \sum_{j=1}^5 \Delta_{w,j}. \quad (43)$$

Let

$$s_{\Delta,w} = \sqrt{\frac{1}{4} \sum_{j=1}^5 (\Delta_{w,j} - \bar{\Delta}_w)^2}. \quad (44)$$

We compute a two-sided 95% Student- t confidence interval over the five paired gains:

$$\bar{\Delta}_w \pm t_{0.975,4} \frac{s_{\Delta,w}}{\sqrt{5}}, \quad t_{0.975,4} = 2.776. \quad (45)$$

The pairing is performed at the replay level; we do not construct independent confidence intervals for RAVEL and the comparator and subtract them. The five paired gains (in percentage points) are LMSYS: {3.2, 2.4, 8.0, 0.8, 6.0}, Burst: {2.4, 7.2, 9.2, 9.4, 9.2}, and DeepResearch: {19.4, 18.6, 11.8, 25.0, 12.2}.

B.3 Repeated-Run Figure and Interpretation

The repeated-run figure plots one point per workload at $\bar{\Delta}_w$ with the 95% Student- t interval from Eq. 45 and a vertical reference line at zero. Each point summarizes five complete paired replays against DualMap.

A positive point estimate establishes that the mean paired difference is positive for those five repetitions. All three

95% intervals in Table 3 lie above zero: LMSYS improves by 4.08 pp on average (95% CI: 0.49–7.67 pp), Burst by 7.48 pp (3.78–11.18 pp), and DeepResearch by 17.40 pp (10.56–24.24 pp). These intervals quantify replay-level variation across the five paired repetitions; they are not formal per-request SLO guarantees.

B.4 Interpretation Boundaries

The result discussion observes the following boundaries:

- The 13.4 pp value is the single-run DeepResearch $16\times$ RAVEL–DualMap difference.
- The 17.40 pp value is the mean RAVEL–DualMap difference across five paired DeepResearch repetitions.
- The two statistics summarize different experimental repetitions and are therefore reported separately.
- Paired confidence intervals quantify replay-level uncertainty for the five-run experiment; they do not convert an empirical result into a formal SLO guarantee.